

Java 学习路线 | 26 年最新零基础到精通一条龙（万人收藏★）

编程导航学习网站：[学编程、做项目、拿 Offer!](#)

企业高频面试题库：[开始刷题，面试遇原题!](#)

精选简历模板大全：[1 分钟搞定简历!](#)

AI 资源导航网站：[获取最新 AI 黑科技!](#)

1 对 1 模拟面试：[随时随地提升面试能力](#)

Java 求职高频面试题：[开始刷题](#)

符号表

可以通过路线知识点前的表情字符，根据自己的实际情况选择学习：

- ○ 所有同学必须学习!!!
- ○ 非常急着找工作，才可不学；目标大厂，必须学习!
- ● 急着找工作的话，可不学；目标大厂，建议学习
- ● 时间充足的话，再去学
- ★ 表示推荐资源

开篇介绍

首先呢，我们要了解 Java 的应用场景和就业方向，看看和自己的学习目的是否一致。

Java 是一门拥有 20 多年历史的老牌编程语言，从诞生之初就凭借着 "一次编写，到处运行" 的特性风靡全球。时至今日，Java 由于优秀的特性、成熟稳定的生态以及庞大的开发者社区，依然是企业级应用开发的首选语言之一。

为什么要学 Java 呢？

很简单，因为 Java 岗位需求量大、就业机会多。无论是后端开发、安卓开发，还是大数据开发，Java 都能胜任。而且 Java 的学习资源非常丰富，视频教程、开源项目应有尽有，非常适合自学。这也是鱼皮当时选择 Java、目前仍然主攻 Java 的原因。

特别是在 AI 时代，Java 程序员的价值不降反升！因为 AI 需要和业务系统深度结合，而 Java 正是最主流的业务开发语言。掌握 Java + AI 应用开发（比如 LangChain4j），将会拥有更强的竞争力。

想了解更多 AI 工具和资源，可以访问鱼皮的 [AI 资源大全](#)。

学好 Java，你能做什么呢？

可以开发企业管理系统、电商平台、社交应用、游戏后台、大数据处理系统，还可以开发 AI 应用（比如智能客服、AI 助手、自动化工具等），应用场景非常广泛。

就业方向

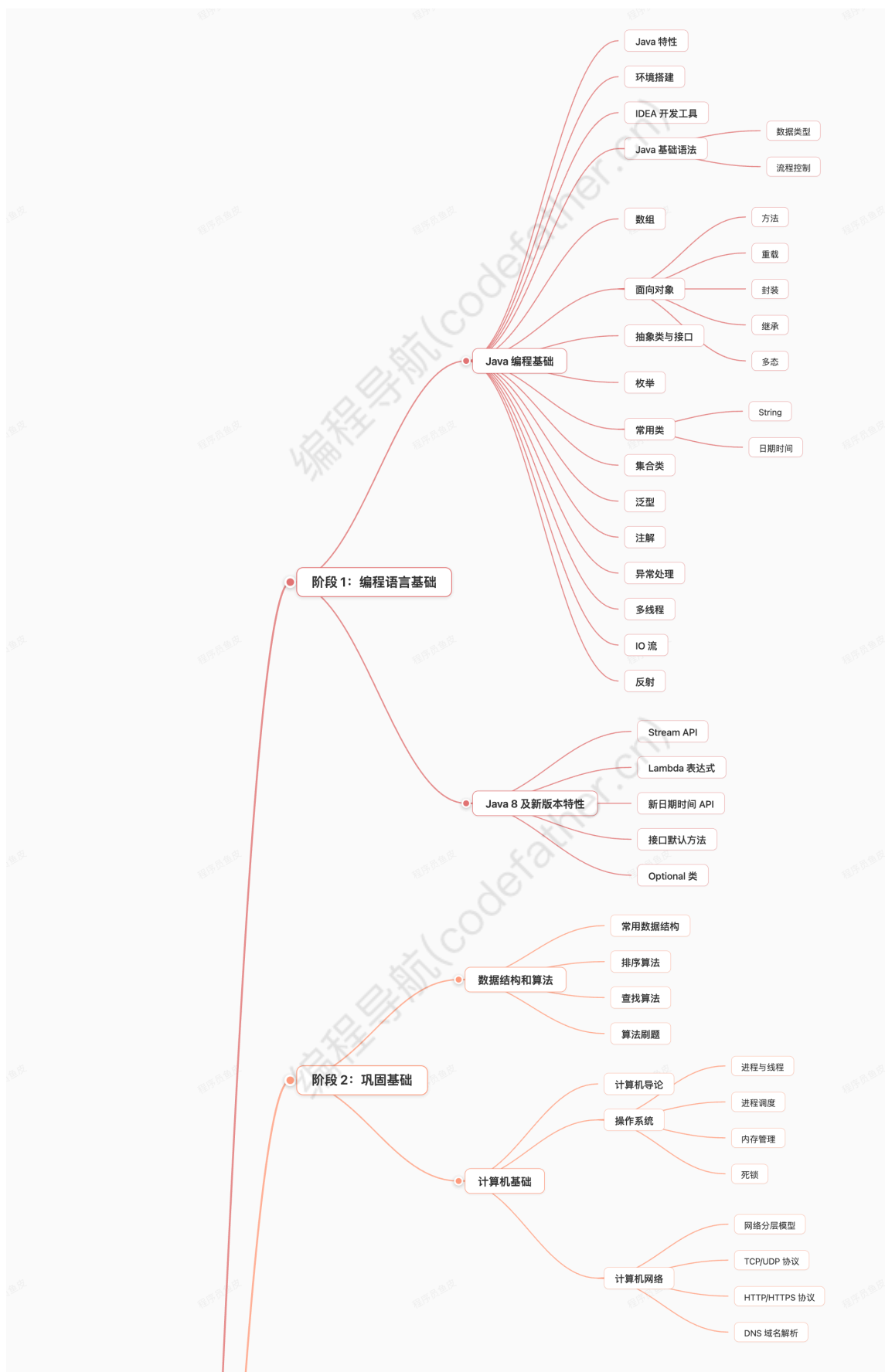
学习 Java 后，常见的就业方向有哪些呢？

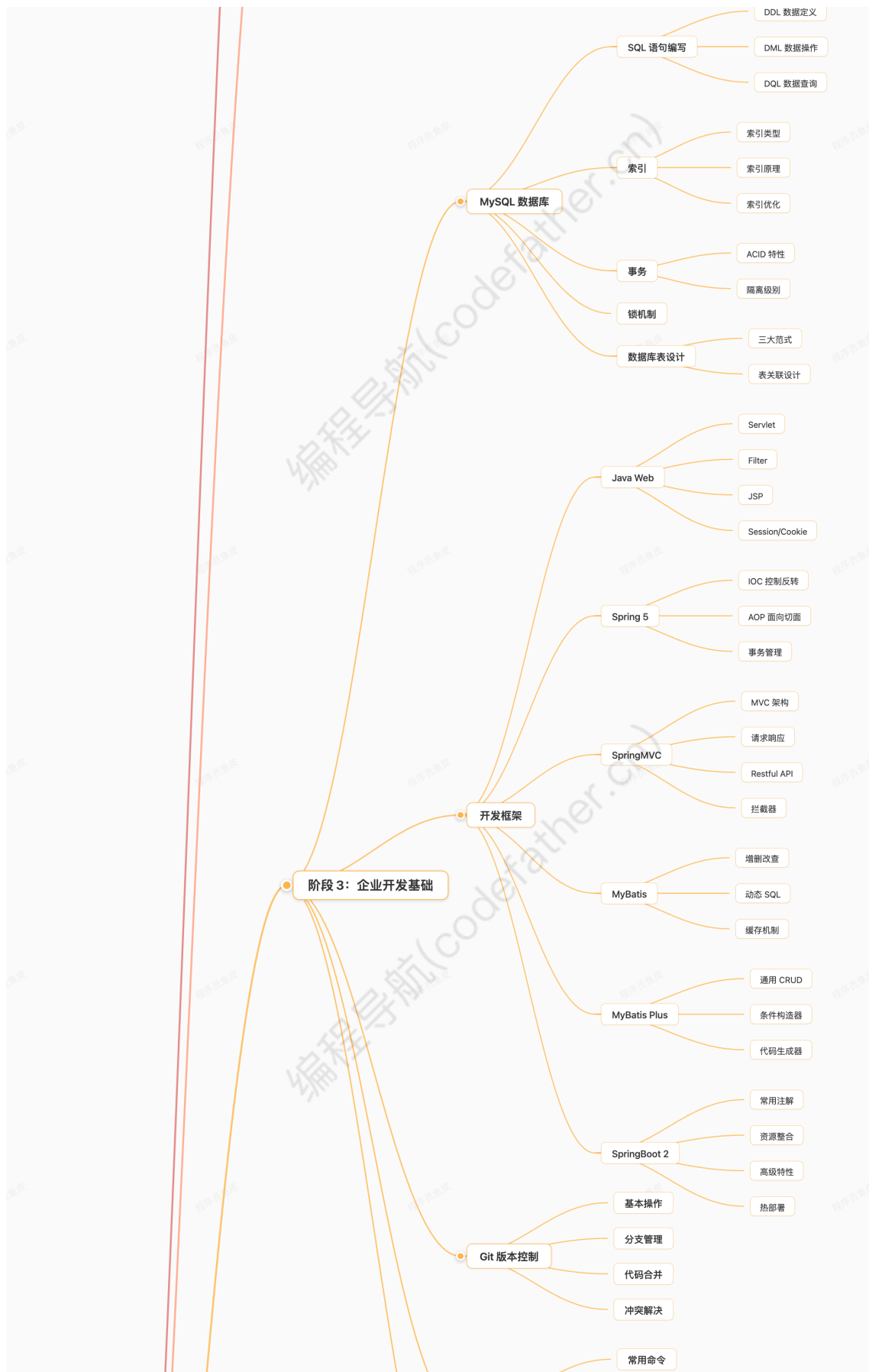
主要包括：

1. ★ Java 后端开发工程师：负责服务端业务逻辑开发，是 Java 最主流的就业方向。工作内容包括设计和实现后端接口、处理业务逻辑、操作数据库、对接第三方服务等。
2. Java 全栈开发工程师：既能做后端也能做前端，能够独立完成整个项目的开发。相比纯后端，需要额外掌握前端技术栈。
3. 安卓开发工程师：使用 Java 或 Kotlin 开发安卓应用，需要额外学习安卓开发框架和移动端知识。
4. 大数据开发工程师：使用 Java 进行大数据处理和分析，需要额外学习 Hadoop、Spark、Flink 等大数据技术栈。
5. 架构师：在有丰富的开发经验后，可以往架构师方向发展，负责系统的整体设计和技术选型。
6. AI 应用开发工程师：在 AI 时代，越来越多的企业需要开发 AI 应用。Java 程序员可以学习 LangChain4j 等 AI 开发框架，结合后端技术开发 AI 应用，这是一个新兴且前景广阔的方向。

学习路线图

下面是完整的 Java 学习路线图，帮助大家理清学习脉络：





Java 学习路线 by 程序员鱼皮

阶段 4：企业开发进阶

Nginx

- 正向代理
- 反向代理
- 负载均衡
- 动静分离
- 集群搭建

微服务概念

Spring Cloud

- 服务注册发现
- Ribbon 负载均衡
- Feign 服务调用
- Hystrix 熔断降级
- Gateway 网关

消息队列

- 消息队列作用
 - 生产消费模型
 - 交换机模型
- RabbitMQ
 - 死信队列
 - 延迟队列
- Kafka
- RocketMQ

Redis 缓存

- 数据类型
- 常用操作
- Java 操作 Redis
 - 数据共享
- 缓存应用场景
 - 分布式锁
 - 限流
- 缓存常见问题
 - 缓存雪崩
 - 缓存击穿
 - 缓存穿透

设计模式

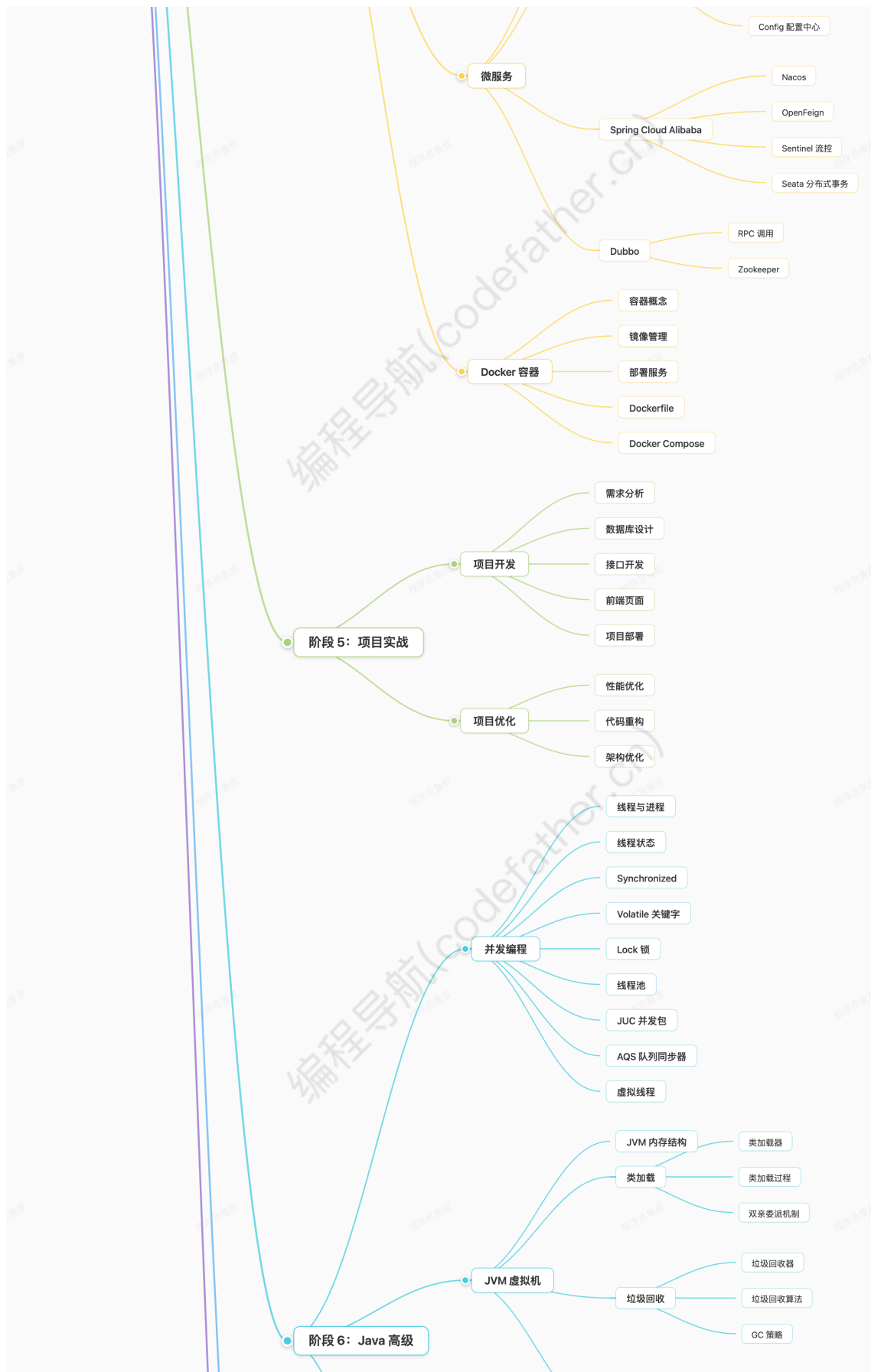
- 创建型模式
 - 单例模式
 - 工厂模式
 - 建造者模式
- 结构型模式
 - 适配器模式
 - 代理模式
 - 装饰器模式
- 行为型模式
 - 策略模式
 - 模板方法模式
 - 观察者模式

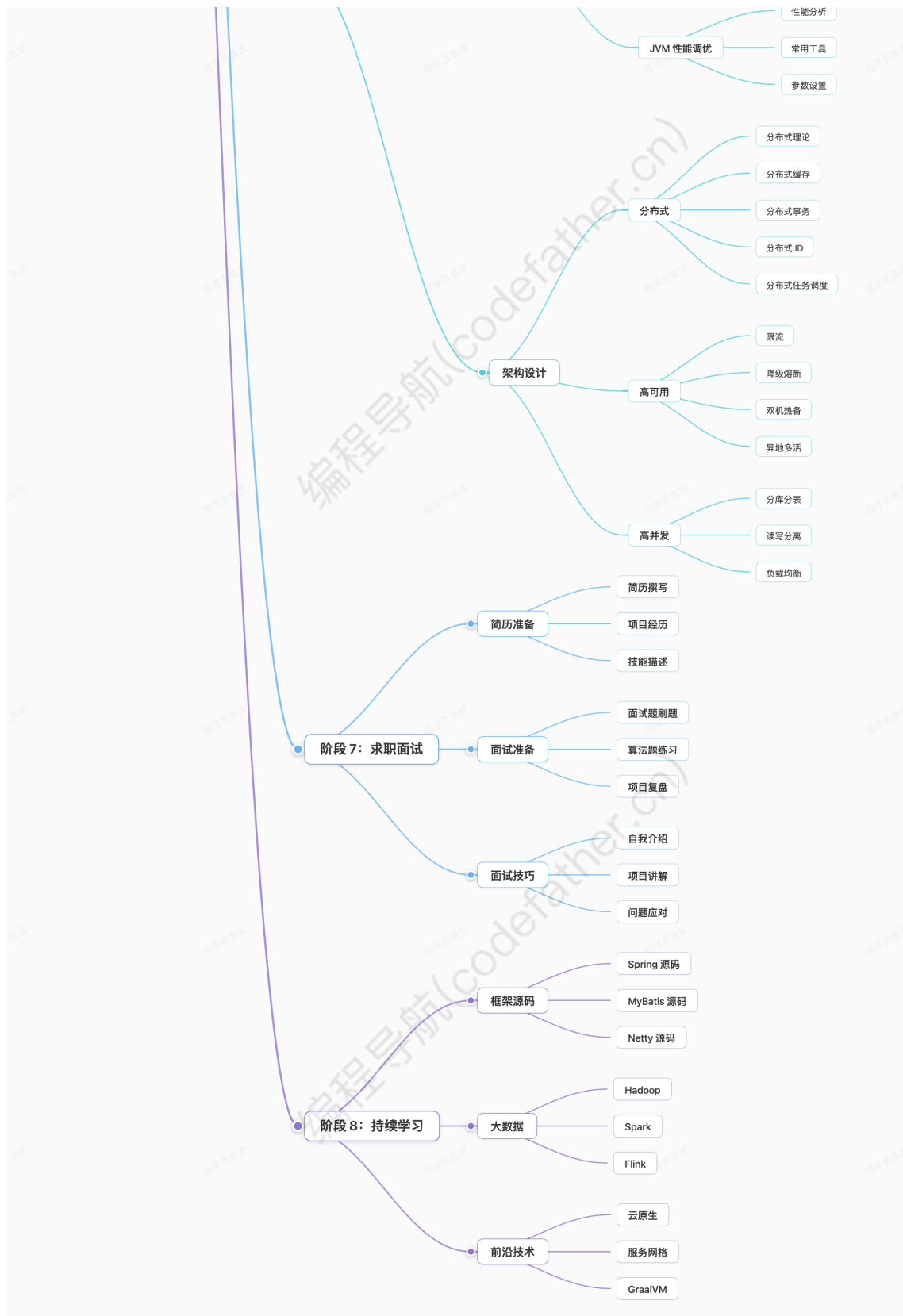
前端基础

- HTML/CSS
- JavaScript
- Ajax
- Vue 框架

Linux 服务器

- 环境搭建
- Shell 脚本
- VIM 使用





阶段 1: 编程语言基础

编程导航(codefather.cn)

编程导航(codefather.cn)

目标

培养兴趣、快速上手，能运行和编写简单的 Java 程序。

学完本阶段后，可以试着用 Java 解决一些数学计算问题、编写图书管理系统等桌面端 GUI 程序，甚至是五子棋之类的小游戏。

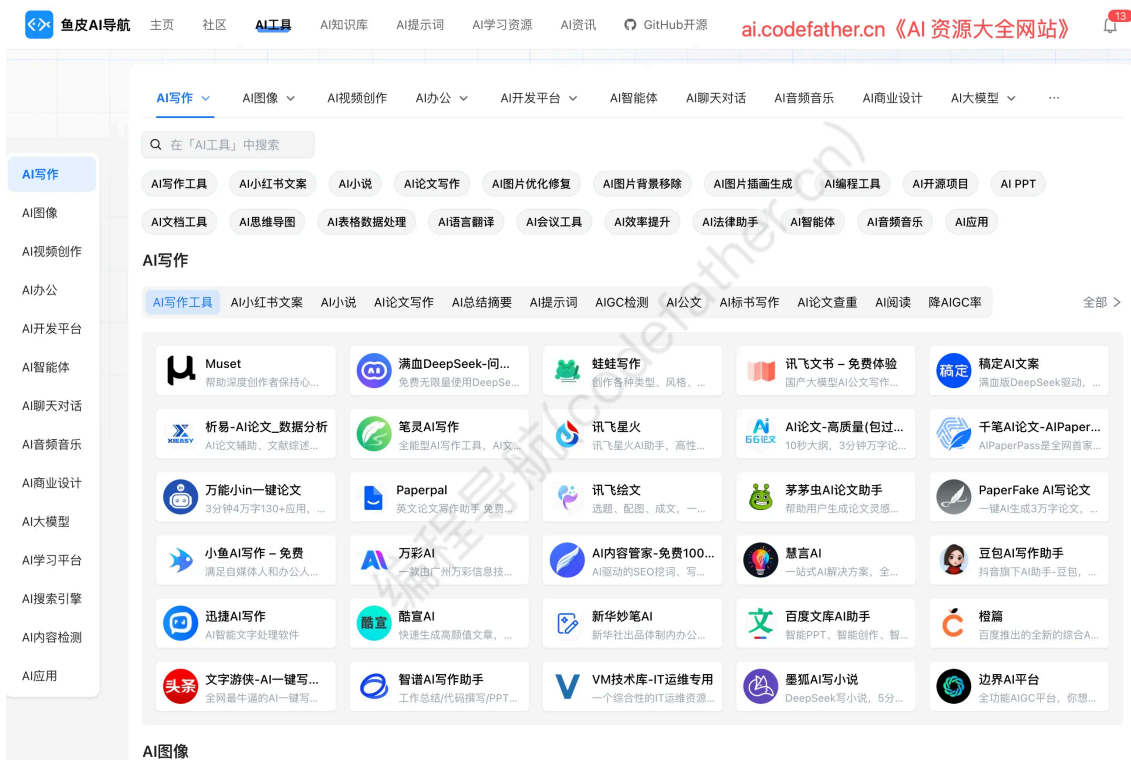
○ Java 编程基础 (45 天)

知识

- Java 特点（看不懂没事，别背！）
- 环境搭建
- IDEA 开发工具
 - 新建项目
 - 运行调试
 - 界面配置
 - 插件管理
- **Java 基础语法**
 - 数据类型
 - 流程控制
- 数组
- **面向对象**
 - 方法
 - 重载
 - 封装
 - 继承
 - 多态
- 抽象类
- 接口
- 枚举
- 常用类
 - String
 - 日期时间
- 集合类
- 泛型
- 注解
- 异常处理
- 多线程
- IO 流
- 反射

学习建议

- 1) 坚持每天学习：初学一门语言时，一定要持续学习，不能中断！建议每天至少学习 2-3 小时，保持学习的连贯性。我当时学 Java 基础的时候，就是每天雷打不动地学，一个半月就把基础过了一遍。
- 2) 多敲代码，多动手：想要学好编程，一定要多敲代码！千万不要只看视频不动手，那样永远学不会。建议先跟着视频或书上的例子敲一遍代码，然后试着自主编写代码，并完成课后练习。很多编程导航的鱼友刚开始学的时候也会犯这个错误，看完视频觉得都懂了，一写代码就懵了。
- 3) 学会使用 Debug：不理解代码也没关系，可以学习 Debug 后，一行一行地打断点执行，查看程序的执行过程。千万不要觉得麻烦，养成习惯后真的能节省很多重复学习的时间。遇到 Bug 不要慌，学会看报错信息，学会用搜索引擎查找解决方案。
- 4) 善用 AI 工具：这是 AI 时代，新时代的程序员一定要学会用 AI 工具！遇到不懂的概念，可以问 AI；遇到 Bug，可以让 AI 帮你分析；甚至可以让 AI 帮你写一些简单的代码。推荐使用鱼皮的 [AI 资源大全](#)，里面有各种好用的 AI 编程工具。



但是记住，AI 是辅助工具，不能完全依赖，自己动手写代码还是最重要的。

5) 不要死记硬背：Java 的语法和 API 非常多，不要试图全部记住。理解为主，多用几次自然就记住了，忘了就查文档。重点是理解编程的思想和解决问题的方法。

经典面试题

1. 为什么重写 equals 还要重写 hashCode?
2. == 和 equals 比较的区别
3. 为啥有时会出现 $4.0 - 3.6 = 0.40000001$ 这种现象?
4. final 关键字的作用
5. 介绍 Java 的集合类
6. ArrayList 和 LinkedList 的区别

更多 Java 基础面试题，推荐到面试鸭刷题：

- [Java 基础面试题](#)
- [Java 集合面试题](#)

资源

- [★ 韩顺平 - 零基础 30 天学会 Java](#)：900 多集，顺序安排很合理，每个知识真正的打碎了，通俗、有示例、有实战、有思想
- [廖雪峰 Java 教程](#)：图文教程，讲解清晰
- [IDEA 中文教程](#)：可以作为课外书来看
- [IDEA Mac 快捷键指南](#)
- [IDEA Win 常用快捷键](#)
- 《Head First Java》：经典入门书籍，适合零基础
- [Codegym 在线游戏](#)：玩玩前几关培养兴趣不错，但后面收费了，不太建议用

🕒 Java 8 及新版本特性 (3 天)

知识

Java 8 核心特性【必学】:

- Stream API
- Lambda 表达式
- 新日期时间 API
- 接口默认方法
- Optional 类

Java 新版本特性【了解】:

随着 Java 的不断发展,从 Java 9 到最新的 Java 25,每个版本都引入了不少新特性。虽然企业中 Java 8 和 Java 11 仍是主流,但了解新版本的特性能让你在技术选型和代码优化时有更多选择。

比较重要的新特性包括:

- Java 17: 记录类 (Record)、密封类 (Sealed Classes)、模式匹配增强
- Java 21 (LTS 长期支持版): 虚拟线程 (Virtual Threads)、序列集合、字符串模板
- Java 21 之后: 更多语法糖和性能优化

感兴趣的同学可以查看鱼皮整理的 [Java 8 ~ 25 新特性大纲](#),里面详细介绍了每个版本的重要特性。

学习建议

Java 8 是如今企业开发中最主流的 Java 稳定版本,在这个版本出现了很多实用的新特性,虽然面试考点不多,但能够提升编程效率,建议学习。Lambda 表达式和 Stream API 真的能让你的代码简洁优雅很多,强烈建议多练习。

此外,很多同学不怎么在简历上写自己会 Java 8,因此如果你把 Java 8 的知识点写在简历上,会大大加分的。

对于新版本特性,刚入门的同学先不用着急学,等你掌握了 Java 8 的核心知识、做过几个项目后,再回过头来了解新特性会更好。不过如果你有充足的时间,提前了解一下也没坏处,特别是虚拟线程 (Virtual Threads) 这种能显著提升并发性能的特性。

经典面试题

1. Java 8 有哪些新特性?
2. Lambda 表达式的原理是什么?
3. Stream API 和传统的循环有什么区别?
4. HashMap 在 JDK 1.7 和 1.8 的区别?
5. Optional 类有什么作用,如何使用?

资源

- ★ [宋红康 - 全网最全 Java 零基础入门教程](#): 只看 Java 8 部分即可
- ★ [鱼皮 - Java 8 ~ 24 新特性讲解](#): 全面讲解各版本新特性
- [鱼皮 - Java 25 新特性讲解](#): 最新版本特性
- ★ [Java 新特性大纲 - 编程导航](#)
- 《Java 8 实战》: 经典书籍
- ★ [在线编写运行 Java 8](#)
- [Java 8 小代码片段](#)

练手项目

- [一本糊涂账](#) (记账系统)
- [Java GUI 图书馆管理系统](#)

- [Java 坦克大战小游戏](#)
- [Swing 俄罗斯方块](#)
- [小小记账本](#) (适合了解数据库的同学)

尾声

学完了 Java 基础后，有些同学会感到迷茫了啊：感觉好像啥也做不出来，不知道下一步做什么，我这一身的本领该如何施展啊？

不要慌，也不要急着去学新技术，接下来我们要多用 Java 来写代码了，巩固基础，但是写什么呢？

当然是数据结构和算法！

阶段 2：巩固基础

注意！如果你时间不够（比如只有 6 个月左右），只是想快速找到工作，那么本阶段甚至可以完全跳过，先去学习开发框架做项目，后面再慢慢弥补基础即可。但如果你时间充裕，或者目标是大厂，建议好好学这个阶段的内容。

目标

想学好编程，计算机基础知识要学好。虽然短期内看不出效果，但长期来看，基础扎实的人走得更远、更稳。

比如算法，是程序员的灵魂。学好算法有助于我们理解程序、开拓思路，因此也是很多公司（尤其是大厂）面试时考察的关键。在找工作前，刷个上百道算法题目是很有必要的。

我们这个阶段的目标是：

1. 熟练使用 Java 语言来编写程序，巩固 Java 基础（直接用 Java 来写算法题目，一举两得，岂不美哉？）
2. 了解计算机基础知识，为后续学习打好基础

此外，建议大家利用零碎时间多去了解 **计算机基础知识**，比如操作系统、计算机网络等。虽然这些知识短期内可能用不上，但对你后面学习开发框架、理解系统架构都有很大帮助。我当时就是因为计算机基础学得比较扎实，学框架的时候理解起来特别快。

Java 基础（30 天）

学习建议

建议大家去阅读《Java 核心技术卷 1》，这本书堪称经典，是帮助你复习巩固 Java 的不二之选，其中图形界面章节可以选择不看。

之后可以刷网上免费的 Java 练习题，检验自己的水平，我当时刷了两遍 1000 题（每天 30 题，1 个月也就刷完了，二刷会更快！）。虽然都是选择题，但能学到很多 Java 语言的特性、避免写代码时容易犯的错误。

资源

- ★ [《Java 核心技术卷 1》](#)：提取码: u74e，经典书籍，必读

🕒 数据结构和算法

数据结构和算法是程序员的基本功，也是大厂面试的必考内容。

详细的数据结构和算法学习路线请参考：[数据结构和算法学习路线](#)

建议结合鱼皮的 [算法学习动画教程](#) 进行学习，用动画的方式讲解算法，通俗易懂。



● 计算机导论

详情请参考：[计算机基础学习路线](#)

知识

- 计算机发展历史
- 计算机应用领域
- 计算机发展方向
- 计算机基本组成
- 二进制
- 编程语言发展

学习建议

大学计算机专业的同学一般刚开学就会上这门课，虽说学习它并不会直接提高你的编程技能，但能够让你更了解计算机和编程，从而在一定程度上帮助你培养学习兴趣、确定学习方向。

自学的话，不用刻意去学习计算机导论，而是可以通过看视频、阅读课外读物的方式慢慢地了解计算机的故事。

资源

- ★ [《计算机科学速成课》](#)：从底层到上层的计算机知识科普，强烈推荐！
- [《半小时漫画计算机》](#)：轻松有趣的计算机入门书

○ 操作系统

详情请参考学习路线：[操作系统学习路线](#)

知识

- 操作系统的组成
- 进程、线程
- 进程 / 线程间通讯方式
- 进程调度算法

- 进程 / 线程同步方式
- 进程 / 线程状态
- 死锁
- 内存管理
- 局部性原理

学习建议

说实话，操作系统这一块知识挺枯燥的。你说说我项目都不会做，你又让我看这些理论，是不是想让我头秃？

我的建议是，可以先利用课余时间看一些网课或者有趣的课外书，对一些操作系统的概念先有个大致的印象，比如进程、线程、死锁，等后面有时间了再系统学习、等到找工作了再去背相关八股文。

还在校园就跟着学校的进度学习就成，自学的话可以看下《清华操作系统原理》视频，有实力的小伙伴，能看懂大黑书就更好了，但如果看不懂也别担心，这并不影响你后续知识的学习。

经典面试题

1. 什么是死锁？死锁产生的条件？
2. 线程有哪几种状态？
3. 有哪些进程调度算法？
4. 什么是缓冲区溢出？

资源

- [《清华操作系统原理》](#)：经典课程
- 《编码》：深入浅出的好书
- 《30 天自制操作系统》：有趣的实践书籍
- 《现代操作系统》：难度较大，不推荐新手看
- 《深入理解计算机系统》：难度较大，不推荐新手看
- 《自己动手写操作系统》：国产好书，网上可以下载
- [浙大操作系统课件](#)

○ 计算机网络

详情请参考学习路线：[计算机网络学习路线](#)

知识

- 网络分层模型
- 网络传输过程
- IP、端口
- HTTP / HTTPS 协议
- UDP / TCP 协议
- ARP 地址解析协议
- 网络安全
- DNS 域名解析

学习建议

很多学习 Java 开发的同学最后都是从事 **后端开发** 的工作，而计算机网络知识是后端开发的重点。

和操作系统一样，自学网络可能会很枯燥，建议先看有趣的课外书，比如《图解 HTTP》；或者有趣的视频，比如《计算机网络微课堂》。后面要找工作面试前，再重点去背一些八股文就好了。还在学校同学好好上课一般就没问题。

学习基础能帮助自己今后发展更稳定，且更容易接受新知识，所以请不要相信基础无用论。

经典面试题

1. 计算机网络各层有哪些协议？
2. TCP 和 UDP 协议的区别？
3. TCP 为什么需要三次握手和四次挥手？
4. HTTP 和 HTTPS 协议的区别？

资源

- [《计算机网络微课堂》](#)：生动有趣的网络课程
- ★ 《图解 HTTP》：经典入门书籍
- 《网络是怎样连接的》：通俗易懂
- ★ 《图解 TCP / IP》：深入理解网络协议
- [浙大计算机网络课件](#)

尾声

巩固基础要花至少 1 个月的时间，当你读完《Java 核心技术卷1》并且不用查询文档也能熟练地用 Java 做题时，就可以接着往下了。

阶段 3：企业开发基础

目标

面向薪资编程，学习实际后台开发工作要用的基础技术和框架，并能 **独立** 做出一个具有完整功能的 Java Web 项目。

这个阶段是从 "初学者" 到 "开发者" 的关键转变，学完这个阶段后，你应该已经能独立开发出大多数常见的后台系统了，比如各种管理系统、商城系统、博客网站等。

○ MySQL 数据库（7 天）

★ 推荐观看 [鱼皮的 MySQL 数据库导学视频](#)，快速了解 MySQL 数据库学习路线和关键知识。

详情请参考学习路线：[数据库学习路线](#)

企业中大部分业务数据都是用关系型数据库存储的，因此数据库是后台开发同学的必备技能，其中 MySQL 数据库是目前的主流，也是面试时的重点。

知识

- 基本概念
- MySQL 搭建
- **SQL 语句编写【必学】**
 - DDL（数据定义语言）
 - DML（数据操作语言）
 - DQL（数据查询语言）
 - DCL（数据控制语言）
- 约束
 - 主键约束
 - 外键约束
 - 唯一约束
 - 非空约束
- **索引【必学】**
 - 索引类型
 - 索引原理
 - 索引优化

- 事务【必学】
 - ACID 特性
 - 隔离级别
- 锁机制【建议学】
 - 行锁 / 表锁
 - 乐观锁 / 悲观锁
- 设计数据库表【必学】
 - 三大范式
 - 表关联设计
- 性能优化【建议学】

学习建议

数据库是后端开发的基石，一定要学扎实！其中，**SQL 语句编写** 和 **设计数据库表** 这两个能力一定要有！

比如让你做一个学生管理系统，你要能想到需要哪些表，比如学生表、班级表；每个表需要哪些字段、字段类型；表之间是什么关系（一对多、多对多）。这些看似简单，但很多同学在实际做项目时就懵了。

建议结合鱼皮的 [SQL 自学网](#) 多写 SQL、多练习，熟能生巧。看完视频后，一定要自己动手设计几个数据库表，比如设计一个博客系统、设计一个电商系统的库表结构。

 SQL之母

[学习](#) [关卡](#) [广场](#) [编程导航](#) [面试题](#) [关于作者](#) [代码开源, 欢迎 star](#)

sqlmothereyupi.icu 免费 SQL 学习, 边学边练, 快乐闯关学 SQL!

基础语法 - 查询 - 全表查询

教程

在 SQL 学习的起点, 我们需要了解一些基本概念, 包括数据库、数据表、SQL、select 查询以及全表查询。

数据库是存放专门存放和管理数据的库。数据库就好比是一座大型的图书馆, 而这个图书馆可以存储大量的信息。我们可以在图书馆中建立各种各样的书架, 每个书架代表一个数据表。

书架上会有很多本书, 数据表中的每一行就相当于一本书, 每一列就相当于这本书的属性, 比如书的名称、书的出版日期等等。

SQL (Structured Query Language) 是一种用于管理、操作和查询数据库的标准化语言, 被广泛应用于各种类型的数据库, 如 MySQL、PostgreSQL、Oracle、Microsoft SQL Server 等, 本系列教程中, 我们将以通用的 SQL 语法带大家入门 SQL 查询的学习。

SQL 的语法是简单易学的, 它使用类似自然语言的结构, 方便开发人员和数据库管理员进行数据库操作和管理。无论是网站应用、企业软件还是大型数据系统, SQL 都是数据库操作的基础和核心。

如何使用 SQL 从数据库中查询数据呢?

首先要了解 select 查询, 就好比是要从图书馆中找到我们感兴趣的书籍。我们可以使用 select 查询从数据表中检索所需的信息, 就像是通过图书馆目录找到了我们想谈的书。

select 查询语句有非常多的语法, 本节我们学习的是最简单直接的 **全表查询**。

当我们使用 select * from 表名 这样的 SQL 语句时, 就是在进行全表查询, 它会返回数据表中的所有行, 让我们可以全面了解表中的数据。

示例

```
1 -- 请在此处输入 SQL
2 select * from student
```

运行

格式化

重置

查看执行结果

执行结果 ✔ 正确

id	name	age	class_id	score	exam_num
1	鸡哥	25	1	2.5	1
2	鱼皮	18	1	400	4
3	热dog	40	2	600	4
4	摸FISH		2	360	4
5	李阿巴	19	3	120	2
6	老李	56	3	500	4

记不住 SQL 语法很正常, 用的时候查文档就行, 但是一定要理解 SQL 的执行逻辑, 理解索引、事务这些核心概念, 这些在面试时经常会被问到。

经典面试题

1. MySQL 索引的最左原则
2. InnoDB 和 MyISAM 引擎的区别?
3. 有哪些优化数据库性能的方法?
4. 如何定位慢查询?
5. MySQL 支持行锁还是表锁? 分别有哪些优缺点?

资源

- [★ 2024 黑马 MySQL 教程](#)：倾向于速成，初学只看完 P57 节前的基础篇即可
- [尚硅谷 - MySQL 基础教程](#)：小姐姐讲课
- [★ 鱼皮的闯关式 SQL 自学网](#)：强烈推荐，边玩边学
- [SQL 在线运行](#)

○ 开发框架 (60 天)

Java 之所以能成为主流的企业开发语言，很大一部分原因是它完善的框架生态，用好框架，不仅能够大大提升开发效率，还能提高项目的稳定性、减少维护成本。

开发框架是后台开发工作中不可或缺的，也是面试考察的重点，一定要好好学！

不知道 Java 能做什么的朋友们，学完开发框架，就会有答案啦。

下面给大家推荐的都是企业中应用最多的主流开发框架，知识点比较零碎，就放在一起讲了。

知识

○ Java Web

- 描述：Java 网页应用开发基础
- 一丢丢前端基础
- XML
- JSON
- Servlet
- Filter
- Listener
- JSP
- JSTL
- Cookie
- Session

○ Spring 5

- 描述：Java 轻量级应用框架
- IOC
- AOP
- 事务

○ SpringMVC

- 描述：Java 轻量级 web 开发框架
- 什么是 MVC?
- 请求和响应
- Restful API
- 拦截器
- 配置
- 执行过程

○ MyBatis

- 描述：数据访问框架，操作数据库进行增删改查等操作
- 增删改查
- 全局配置
- 动态 SQL
- 缓存
- 和其他框架的整合
- 逆向工程

● MyBatis Plus

- 描述：Mybatis 的增强工具，能够简化开发、提高效率
- 引入
- 通用 CRUD
- 条件构造器
- 代码生成器
- 插件扩展
- 自定义全局操作

○ SpringBoot 2

- 描述：简化 Spring 应用的初始搭建以及开发过程，提高效率
- 常用注解
- 资源整合
- 高级特性
- 本地热部署

● Spring Security

- 描述：Spring 的安全管理框架
- 用户认证
- 权限管理
- 相关技术：Shiro

● Maven / Gradle

- 描述：项目管理工具
- 构建
- 依赖管理
- 插件
- 配置
- 子父工程
- 多模块打包构建
- Nexus 私服搭建

学习建议

1) 选择同一系列教程：由于技术较多，且框架之间存在一定的联系，因此建议大家看同一系列的视频教程（比如都看黑马的，或者都看尚硅谷的），以保证学习内容的连续以及体验上的一致。不要今天看这个老师的 Spring，明天看那个老师的 MyBatis，容易混乱。

2) 多记笔记，多敲代码：学这些技术的时候，千万不能懒！一定要多记笔记，并且跟着老师写代码。原理部分不要太过纠结，先以能跟着敲出代码、写出可运行的项目为主，有些东西做出来也能帮助你更好地理解理论。我的建议是，视频倍速播放，但是代码一定要一行一行手敲，不要复制粘贴。

3) 注意学习顺序：学习顺序挺重要的，建议按我推荐的顺序学，不要一上手就学 Spring Boot。只有先学习下自己整合框架的方法（比如手动配置 Spring + MyBatis），才能帮你理解 SpringBoot 解决的问题，感受到它的方便和高效。这样学下来，面试的时候也能说出个所以然来。

4) 框架会用就行：Maven / Gradle 当成工具用就好，面试基本不问，跟着框架教程去用就行了，急着找工作的话，先不用花太多时间去深入学。大厂面试问这个的也不多，会配置、会用就够了。

经典面试题

1. Spring 的 IOC 和 AOP 是什么，有哪些优点？
2. Spring 框架用到了哪些设计模式？
3. 介绍 Spring Bean 的生命周期
4. MyBatis 如何实现延迟加载？

5. 介绍 MyBatis 的多级缓存机制

更多框架面试题：

- [Spring 面试题 - 面试鸭](#)
- [SpringBoot 面试题 - 面试鸭](#)
- [MyBatis 面试题 - 面试鸭](#)

资源

下面的资源分为两大类，希望快速做出项目、快速就业的同学请看【速成视频】；想要系统学习、打好基础的同学可以看【非速成视频】。

速成视频（按顺序看，同类视频任意选择 1 个即可）：

- ★ [2024 黑马 JavaWeb](#)：包含前端、MySQL、Java Web、MyBatis、Spring MVC、Spring、Spring Boot、Maven 等知识，内容很新很全面
- [2024 尚硅谷 SSM + MyBatis Plus 整合学习](#)：比较系统地讲解了 SSM 框架的整合
- [黑马 Spring Boot 2](#)：讲解 SpringBoot 核心特性
- [尚硅谷 Spring Boot 2](#)：内容全面，讲得很细

非速成视频（按顺序看）：

- ★ [尚硅谷 JavaWeb 全套教程](#)：前端部分最好也看下，虽然有点老但基础讲得很扎实
- ★ [尚硅谷 - Spring 5 框架最新版教程 \(IDEA 版\)](#)：系统讲解 Spring 核心思想
- ★ [尚硅谷 - SpringMVC 2021 最新教程](#)：MVC 架构讲得很清楚
- ★ [尚硅谷 - MyBatis 实战教程全套完整版](#)：经典教程，讲得很细
- ★ [尚硅谷 - MyBatisPlus 教程](#)：提高开发效率必学
- [Maven 零基础入门教程](#)：搞不懂 Maven 可以看看
- ★ [雷丰阳 2021 版 SpringBoot2 零基础入门](#)：讲得非常细致全面
- [尚硅谷 - SpringSecurity 框架教程](#)：权限框架必学

学习完框架后，即可跟着鱼皮的原创项目教程系列边学边做项目。用项目驱动学习，更快地掌握后端必学技术，并直接写在简历上：[项目实战 - 鱼皮原创项目教程系列](#)。

● 开发规范（3 天）

开发不规范，同事两行泪。

开发规范是团队开发中必须遵守的，有利于提高项目的开发效率、降低维护成本。

知识

- 代码规范
 - 代码风格
 - 命名
 - 其他规则
- 代码校验 (CheckStyle)
- 提交规范

学习建议

有时间的话，简单过一遍大厂团队的代码规范手册就好了，以后做项目的时候能想起来的话就去使用，或者从书中、网上查规范文档，再去遵守。

项目做得多了，自然会养成好的习惯，不用刻意去记（毕竟每个团队规范也不完全相同，背了也没用）。也可以直接利用开发工具自带的一些代码检查插件，帮忙养成好的编码习惯。

资源

- ★ [阿里巴巴 Java 开发手册](#)：搜索《Java开发手册》，阿里官方代码规范
- [华山版《Java开发手册》独家讲解](#)：视频讲解
- [Google Java Style Guide](#)：谷歌 Java 代码规范

○ Git (3 天)

详情请参考学习路线：[Git & GitHub 学习路线](#)

此前大家可能听说过 GitHub，一流的代码开源托管平台。

Git 和它可不一样，是一个版本控制工具，可以更好地管理和共享项目代码，比如把自己的代码传到 GitHub 上、或者从远程下载。

无论自己做项目、还是团队开发，Git 都是现在不可或缺的神器。

知识

- 区分 Git 和 GitHub
- 工作区
- 分支
- 代码提交、推送、拉取、回退、重置
- 分支操作
- 代码合并、解决冲突
- 标签
- cherry-pick
- Git Flow
- 相关技术：SVN（比较老）

学习建议

每个命令跟着敲一遍，有个大致的印象，会用即可。

建议平时大家可以多把自己的代码使用 Git 命令上传到 GitHub 上，用的多了自然就熟悉了。

经典面试题

1. 如何解决提交冲突？
2. 提交不小心出现误操作，如何撤销？
3. 什么是 Git Flow，它有什么好处？

资源

- ★ [尚硅谷 - 5h 打通 Git 全套教程 | 2021 最新 IDEA 版](#)
- [猴子都能懂的 Git 入门](#)（图文并茂，适合入门）
- ★ [GitHub 漫游指南](#)（GitHub 使用指南）
- [GitHub 官方文档](#)
- [Learning Git Branching](#)（游戏化学习 Git）

○ Linux (10 天)

★ 建议先观看 [鱼皮的 Linux 导学视频](#)，快速了解 Linux 学习路线和关键知识

详情请参考学习路线：[Linux 学习路线](#)

企业中的很多前后台项目都是部署在 Linux 服务器上的，因此很有必要熟悉 Linux 的操作和脚本的编写。

后面学微服务、学架构都是在多台服务器操作，如果你不熟悉 Linux，会有点吃力。

知识

- Linux 系统安装
- 环境变量
- 文件管理
- 用户管理
- 内存管理
- 磁盘管理
- 进程管理
- 网络管理
- 软件包管理
- 服务管理
- 日志管理
- Linux 内核
- 常用命令
- 常用环境搭建
- Shell 脚本编程
- VIM 的使用

学习建议

Linux 是后端开发必备技能，因为企业中的项目基本都是部署在 Linux 服务器上的。

- 1) 动手实践最重要：建议自己购买一台云服务器（阿里云、腾讯云学生机一年才几十块钱），并且在本地用 VMware 搭建 Linux 虚拟机环境。光看视频是学不会 Linux 的，一定要自己敲命令！
- 2) 从零开始部署项目：看完基础教程后，一定要自己从 0 开始手敲命令安装软件（比如 MySQL、Redis、Nginx）、部署服务，熟悉整个项目的上线流程。第一次部署肯定会遇到各种问题，这很正常，解决问题的过程就是最好的学习过程。
- 3) 常用命令要熟悉：每个命令至少要跟着敲一遍，了解它们的作用，并通过自然地练习，熟悉常用的 Linux 命令（cd、ls、cat、grep、ps、top、vi 等）。记不住没关系，用的时候查文档就行，但是一些高频命令要做到能随手写出来。
- 4) 先会用，再理解：不要一上来就去研究 Linux 内核，那样会把自己绕晕。先会用，能用 Linux 部署项目、排查问题就行。一般面试问的 Linux 题目也不会很难，主要考察常用命令和基本概念，面试前去背一下八股文就没什么问题。等工作后有需要了，再去深入学习 Linux 内核设计也不迟。

经典面试题

1. 如何查看某个进程的运行状态？
2. 如何在 Linux 上查看 2 G 的大文件？
3. Linux 软链接和硬链接的区别

资源

- ★ [2021 韩顺平 - 一周学会 Linux](#)：基于 CentOS 7.6 版本
- [《鸟哥的 Linux 私房菜 —— 基础篇》](#)：经典教材，可在线阅读
- [Linux 工具快速教程](#)：基础、工具进阶、工具参考
- ★ [蓝桥云课 - Linux 基础入门](#)：在线实战环境
- [腾讯云动手实验室](#)
- [阿里云体验实验室](#)
- [华为云沙箱实验室](#)
- ★ [Linux 命令搜索](#)：命令速查必备
- [Linux 命令大全手册](#)

- [宝塔 Linux 面板](#)：可视化服务器管理

● 前端基础（14 天）

详情请参考鱼皮原创的 [保姆级前端学习路线](#)。

虽然 Java 程序员面试时基本不会出现前端相关问题，但是在企业中，往往需要前后端程序员配合完成工作。会一些前端，不仅可以提高你们的协作效率，还能提高自己对整个项目的了解和掌控力，甚至能独立开发出一个完整项目！这点也是能给面试加分的。

知识

- HTML
- CSS
- JavaScript
 - Ajax
- Vue

学习建议

作为 Java 后端开发，不需要学习太深的前端技术，但是了解一些基础的前端知识还是很有必要的。

- 1) 学到什么程度：熟悉下基础的前端三件套（HTML、CSS、JavaScript），了解前端是如何向后端发送请求（Ajax）来做数据交互的，一般就够了。不需要深入学习各种前端框架和工程化工具。
- 2) 建议学 Vue：有时间的话可以学下 Vue，是比较容易上手的主流前端开发框架。学会了 Vue，你就能独立开发一个前后端分离的完整项目，Vue + SpringBoot 的组合还是很香的。而且简历上能写 "全栈开发" 也更有竞争力。
- 3) 跟着项目学：推荐跟着鱼皮的原创项目教程系列学习前端，每个项目都是前端 + 后端的全栈项目，边做项目边学前端，比单独看前端教程效果好很多。
- 4) 善用 AI：AI 时代学习前端更简单了！可以让 AI 帮你生成前端代码、解释前端概念、修复 Bug。推荐跟着鱼皮的 [AI 程序员技术练兵场项目](#) 学习如何用 AI 辅助前端开发，能让你的开发效率翻倍。

练手项目

推荐跟着鱼皮的原创项目教程系列边学边做项目，每个项目都是前端 + 后端的全栈项目。用项目驱动学习，更快掌握前端基础和后端必学技术，并直接写在简历上：[项目实战 - 鱼皮原创项目教程系列](#)。

AI 应用开发：

当前 AI 应用开发已成为最热门的技术方向之一！建议在学习传统项目的同时，尝试将 AI 能力融入到你的项目中。比如在管理系统中加入智能推荐、自动生成报表、AI 客服等功能，这类项目在求职时会非常加分。

强烈推荐学习鱼皮的 AI 应用开发系列项目，能够快速掌握 LangChain4j、Spring AI 等主流 AI 开发框架：

- [AI 编程助手](#)：适合新手入门 AI 应用开发，实战 LangChain4j 框架
- [AI 程序员技术练兵场](#)：以 AI 编程为主的 Java + Vue 全栈 AI 应用开发教程
- [AI 零代码应用生成平台](#)：对标大厂的微服务 AI 项目，实战 AI 智能体、LangGraph4j 工作流、微服务架构
- [AI 超级智能体项目](#)：学习实践 AI 应用开发，掌握新时代程序员必知的 AI 技术

其他开源项目：

- HotelSystem：<https://github.com/misterchaos/HotelSystem>（酒店管理系统
Java,tomcat,mysql,servlet,jsp实现，没有使用任何框架）
- 超市管理系统：<https://github.com/zhanglei-workspace/shopping-management-system>
- Mall4j：<https://github.com/gz-yami/mall4j>（Spring Boot 电商系统）
- newbee-mall：<https://github.com/newbee-ltd/newbee-mall>（基于 Spring Boot 2.X 的全栈电商系统）

- litemall: <https://github.com/linlinjava/litemall> (小商城系统, Spring Boot 后端 + Vue 管理员前端 + 微信小程序用户前端 + Vue 用户移动端)
- forum-java: <https://github.com/Qbian61/forum-java>
(一款用 Java Spring Boot 实现的现代化社区系统)

尾声

学完这个阶段的知识后,一定要再串起来回忆一遍,必须自己独立开发一个 Java Web 项目(量级可以不大,但你学过的技术尽可能地用上),能发布到 Linux 服务器上让其他小伙伴访问就更好了~

在 AI 时代,建议大家在掌握基础开发能力后,尽早开始学习 AI 应用开发!现在 AI 应用开发已经成为热门方向,很多企业都在招聘懂 AI 的 Java 开发工程师。建议在做项目时,尝试融入 AI 能力,比如智能推荐、内容生成、AI 对话等功能。鱼皮的 [AI 编程助手](#)、[AI 程序员技术练兵场](#) 等项目都是很好的入门选择,能够帮你快速掌握 Spring AI、LangChain4j 等主流 AI 开发框架。

如果你只是对 Java 感兴趣、或者只是想试着自己开发后台,并不是想靠 Java 找工作的话,学到这里就可以了。可以把更多时间投入到你主方向的学习中。

但如果你是想找 Java 方向的工作,尤其是想进大厂的话,一定要继续努力,用心学习下个阶段的企业开发进阶知识。

阶段 4: 企业开发进阶

目标

学习更多企业级开发技术和编程思想,能够结合多种技术,独立开发出架构合理的完整系统, **解决实际问题**。

前面几个阶段学的技术,已经能让你做出一个基本的系统了。但是在实际的企业开发中,我们还会遇到很多问题:系统访问量大了怎么办?如何提高系统性能?如何保证系统的稳定性?多个系统之间如何协作?这个阶段就是要学习解决这些问题的技术。

要了解为什么需要这个技术?什么时候用这个技术?某个需求该用哪些技术?不要为了用技术而用技术。

当然,这个阶段的内容有些过于丰富,不是所有的东西都要学,大家可以根据自己的实际情况(时间、求职紧迫程度),有选择地学习。如果时间紧急,可以先学 Redis 和微服务,其他的等工作后再补。

● 软件工程

详情请参考学习路线: [软件工程学习路线](#)

软件开发和管理的一些概念、原则、技术、方法、工具和经验。

知识

- 软件的本质
- 软件特性
- 软件过程
- 软件开发原则
 - 开闭原则
 - 里氏替换原则
 - 依赖倒置原则
 - 单一职责原则
 - 接口隔离原则
 - 迪米特法则
- 软件过程模型
- 敏捷开发

- 软件开发模型
- 需求建模
- 软件设计
- UML
- 体系结构设计
- 设计模式
- 软件质量管理
- 评审
- 软件质量保证
- 软件测试
 - 单元测试
 - 集成测试
 - 系统测试
 - 压力测试
 - 部署测试
- 软件配置管理
- 软件项目管理
- 软件项目估算
- 项目进度安排
- 风险管理
- 软件过程改进
 - 成熟度模型

学习建议

大学软件专业的必修课，偏理论，能学到很多企业软件开发的方法，也是对软件开发同学综合能力的提升，有时间的话可以了解下。但对想要快速找工作的同学来说，忽略即可，面试基本不会问。

资源

- [《软件工程：实践者的研究方法》](#)（经典大黑书，讲得非常全面）
- [《软件开发的 201 个原则》](#)（工具书，看一遍挺好的）
- [北京大学软件工程](#)（公开课）
- [大连工业大学软件工程](#)
- [浙大计算机软件工程课件](#)

○ 设计模式（21 天）

★ 推荐观看 [鱼皮的设计模式导学视频](#)，快速掌握设计模式的高效学习法

详情请参考学习路线：[设计模式学习路线](#)

设计模式是软件开发中解决一类问题的通用方法。

使用设计模式能让你写出更优雅、可维护的代码，也正因如此，很多框架源码都用到了设计模式，你不学很难看懂。

此外，鱼皮改了几百份简历，基本上没有同学把设计模式写在项目经历中。**因此学好设计模式并写在简历上是很加分的！**

知识

- 创建型模式：对象实例化的模式，创建型模式用于解耦对象的实例化过程
 - 单例模式
 - 工厂方法模式
 - 抽象工厂

- 建造者模式
- 原型模式
- 结构型模式：把类或对象结合在一起形成一个更大的结构
 - 适配器模式
 - 组合模式
 - 装饰器模式
 - 代理模式
 - 享元模式
 - 外观模式
 - 桥接模式
- 行为型模式：类和对象如何交互，及划分责任和算法
 - 迭代器模式
 - 模板方法模式
 - 策略模式
 - 命令模式
 - 状态模式
 - 责任链模式
 - 备忘录模式
 - 观察者模式
 - 访问者模式
 - 中介者模式
 - 解释器模式

学习建议

设计模式是个好东西，但是很多同学学完了却不知道怎么用，或者用得不对。

- 1) 先理解再实践：先理解概念，了解每个设计模式的特点和应用场景（解决什么问题、什么时候用），再多加练习，运用到实际项目中。千万不要死记硬背 23 种设计模式，那样毫无意义。
- 2) 重点掌握常用的：实际工作中，并不是所有的设计模式都会用到。建议重点掌握单例模式、工厂模式、策略模式、模板方法模式、代理模式这几个最常用的，其他的了解即可。
- 3) 结合项目学习：推荐跟着鱼皮的 [聚合搜索项目](#)、[OJ 判题系统](#) 学习，这两个项目都运用了大量的设计模式（策略模式、工厂模式、模板方法等），能让你理解设计模式在实际项目中是怎么用的。光看书和视频，很难真正理解设计模式的精髓。
- 4) 写在简历上：鱼皮改了几百份简历，基本上没有同学把设计模式写在项目经历中。如果你的项目中用到了设计模式，一定要写在简历上，这是很大的加分项！比如 "使用策略模式 + 工厂模式实现了多种文件上传方式的灵活切换"。

经典面试题

1. 单例模式有哪些实现方式？有哪些优缺点？请手写其中一种
2. 你用过哪些设计模式，为什么用它？
3. 工厂模式和抽象工厂模式的区别？
4. 代理模式的应用场景有哪些？
5. 策略模式如何消除 if-else？

资源

- ★ [鱼皮的设计模式教程](#)：GitHub 免费开源
- ★ [聚合搜索项目 - 鱼皮原创](#)：运用了大量设计模式
- ★ [OJ 判题系统 - 鱼皮原创](#)：运用了多种设计模式
- ★ 《图解设计模式》：强烈推荐，用 Java 语言实现，图多、有示例代码、有习题和答案

- 《Head First 设计模式》：经典入门书籍
- 《大话设计模式》：通俗易懂
- 《设计模式：可复用面向对象软件的基础》：大黑书，有能力的话也可以去读
- [尚硅谷图解 Java 设计模式](#)
- [图说设计模式](#)

○ Redis (14 天)

★ 推荐观看 [鱼皮的 Redis 导学视频](#)，快速了解 Redis 学习路线和关键知识。

详情请参考学习路线：[Redis 学习路线](#)

缓存是高并发系统不可或缺的技术，可以提高系统的性能和并发，而 Redis 是实现缓存的最主流技术，因此它是后台开发必学的知识点，也是面试重点。

知识

- Redis 基础
- 什么是缓存？
- 本地缓存
 - Caffeine 库
- 多级缓存
- Redis 分布式缓存
 - 数据类型
 - 常用操作
 - Java 操作 Redis
 - Spring Boot Redis Template
 - Redisson
 - 主从模型搭建
 - 哨兵集群搭建
 - 日志持久化
- 缓存（Redis）应用场景
 - 数据共享
 - 单点登录
 - 计数器
 - 限流
 - 点赞
 - 实时排行榜
 - 分布式锁
- 缓存常见问题
 - 缓存雪崩
 - 缓存击穿
 - 缓存穿透
 - 缓存更新一致性
- 相关技术：Memcached、Ehcache

学习建议

学会如何简单地使用缓存并不难，和数据库类似，无非就是调用 API 对数据进行增删改查。但是要用好 Redis，还是需要下一番功夫的。

建议的学习路径是：先能够独立使用 Redis，了解缓存的应用场景（比如缓存热点数据、实现分布式锁、计数器等）；再学习如何在 Java 中操作 Redis（推荐使用 Spring Boot + Redis 或 Redisson），并尝试把 Redis 应用到你之前做过的

项目中，实际感受一下 Redis 带来的性能提升。

跟着视频教程实操一遍即可，Redis 的一些高级特性和底层原理，可以等到面试前再去深入了解。但是缓存三大问题（缓存穿透、击穿、雪崩）一定要理解，因为面试必问！

推荐大家看黑马的那套 Redis 教程，讲得非常好，结合实际项目去讲，而不是纯理论。看完之后，一定要自己动手把 Redis 加到你的项目里，哪怕只是用 Redis 缓存一下用户信息也好。

经典面试题

1. Redis 为什么快？
2. Redis 有哪些常用的数据结构？
3. Redis RDB 和 AOF 持久化的区别，如何选择？
4. 如何解决缓存击穿、缓存穿透、雪崩问题？
5. 如何用 Redis 实现点赞功能，怎么设计 Key / Value？

资源

- ★ [伙伴匹配系统 - 鱼皮原创](#)：项目中运用 Redis 解决问题
- ★ [智能 BI 项目 - 鱼皮原创](#)：运用 Redis
- ★ [黑马 Redis 从基础到原理实战教程](#)：结合项目去讲，内容全面，强烈推荐
- [尚硅谷 - 2021 最新 Redis 6 入门到精通教程](#)：基于 Redis 6，讲解详细
- ★ [Redis 完整学习路线 - 编程导航](#)
- 《Redis 实战》：经典书籍

消息队列（14 天）

★ 推荐观看 [鱼皮的消息队列 RabbitMQ 导学视频](#)，快速了解消息队列学习路线和关键知识。

关于消息队列的详细学习，可以查看 [消息队列学习路线（通用）](#)

消息队列是用于传输和保存消息的容器，也是大型分布式系统中常用的技术，主要解决应用耦合、异步消息、流量削峰等问题。后台开发必学，也是面试重点。

知识

- 消息队列的作用
- RabbitMQ 消息队列
 - 生产消费模型
 - 交换机模型
 - 死信队列
 - 延迟队列
 - 消息持久化
 - Java 操作
 - 集群搭建
- 相关技术：Kafka、ActiveMQ、TubeMQ、RocketMQ

学习建议

消息队列听起来很高大上，但其实学会如何使用并不难，无非就是调用 API 去生产、转发和消费消息。难的是理解什么时候该用消息队列，以及如何设计好消息队列的使用方案。

建议先能够独立使用消息队列，了解它的应用场景（比如异步处理、流量削峰、系统解耦）；再学习如何在 Java 中操作消息队列中间件（使用 Spring Boot 整合），并尝试把消息队列应用到你的项目中，实际感受一下它带来的好处。

选择哪个消息队列呢？

市面上消息队列很多（RabbitMQ、Kafka、RocketMQ 等），初学者建议先学习 RabbitMQ，比 Kafka 要好理解一些，而且企业中用得也很多。

- 关于 RabbitMQ 的详细学习，可以查看 [RabbitMQ 消息队列学习路线](#)
- 关于 Kafka 的详细学习，可以查看 [Kafka 消息队列学习路线](#)
- 关于 RocketMQ 的详细学习，可以查看 [RocketMQ 消息队列学习路线](#)

跟着视频教程实操一遍即可，原理和高级特性可以等到面试前再去深入了解。

推荐跟着鱼皮的 [智能 BI 项目](#) 或者 [OJ 判题系统](#) 学习，这两个项目都用到了消息队列解决实际问题（BI 项目用消息队列异步生成图表，判题系统用消息队列解耦），能让你理解消息队列在真实项目中是怎么用的。

经典面试题

1. 使用消息队列有哪些优缺点？
2. 如何保证消息消费的幂等性？
3. 消息队列有哪些路由模型？
4. 你是否用过消息队列，解决过什么问题？

更多消息队列面试题：

- [消息队列面试题 - 面试鸭](#)
- [RabbitMQ 面试题 - 面试鸭](#)
- [RocketMQ 面试题 - 面试鸭](#)

资源

- ★ [智能 BI 项目 - 鱼皮原创](#)：项目中运用消息队列异步生成图表
- ★ [OJ 判题系统 - 鱼皮原创](#)：运用消息队列削峰
- ★ [2024 黑马 RabbitMQ 消息队列教程](#)：适合快速入门
- [尚硅谷 - 2021 最新 RabbitMQ 教程](#)：更加全面
- ★ [编程导航原创 Rocket MQ 消息队列专栏](#)
- [RabbitMQ 中文文档](#)
- 《RabbitMQ 实战：高效部署分布式消息队列》：经典书籍
- ★ [RabbitMQ 在线模拟器](#)

○ Nginx（14 天）

★ 推荐观看 [鱼皮的 Nginx 导学视频](#)，快速了解 Nginx 学习路线和关键知识

关于 Nginx 的详细学习，可以查看 [Nginx 学习路线](#)

Nginx 是主流的、开源的、高性能的 HTTP 和反向代理 web 服务器，可以用于挂载网站、请求转发、负载均衡、网关路由等。前后端开发同学都需要学习，在后端开发的面试中有时会考到。

知识

- Nginx 作用
- 正向代理
- 反向代理（负载均衡）
- 常用命令
- 配置
- 动静分离（网站部署）
- 集群搭建
- 相关技术：HAProxy、Apache

学习建议

Nginx 是个非常实用的工具，学会了你就能自己搭建网站、做负载均衡了。

- 1) Nginx 的基本使用非常简单，甚至不需要看视频教程，跟着一篇文章或者官方文档就能够用它来部署网站、实现反向代理。鱼皮之前录过一个 [手把手搭建个人网站视频教程](#)，10 分钟就能学会基本操作。
- 2) 实践出真知：建议自己买台服务器，实际部署几个网站试试。比如把你之前做的项目部署到服务器上，用 Nginx 做反向代理和负载均衡。实际操作一次，胜过看十遍理论。
- 3) 要理解 Nginx 的配置：虽然 Nginx 配置看起来很简单，但是在企业中，往往需要针对实际场景去写一些特定的配置（比如配置 HTTPS、配置缓存、配置限流等）。因此建议有时间的话，多实践 Nginx 的各种配置，了解 Nginx 的设计思想，对今后自己设计系统时也有帮助。

经典面试题

1. Nginx 有哪些作用？
2. Nginx 为什么支持高并发？
3. Nginx 有哪些负载均衡策略？
4. 什么是 Nginx 惊群问题，如何解决它？

更多 Nginx 面试题：

- [Nginx 面试题 - 面试鸭](#)
- [Nginx 原理面试题 - 面试鸭](#)
- [Nginx 配置面试题 - 面试鸭](#)

资源

- ★ [尚硅谷 - Nginx 教程由浅入深](#)：讲得比较全面
- ★ [鱼皮 - 手把手从 0 搭建个人网站](#)：简单演示 Nginx 部署网站
- [Nginx 配置在线生成](#)

● Netty 网络编程（21 天）

关于 Netty 网络编程的详细学习，可以查看 [Netty 网络编程学习路线](#)

开源的 Java 网络编程框架，用于开发高性能（事件驱动、异步非阻塞）、高可靠的网络服务器和客户端程序。

很多网络框架和服务器程序都用到了 Netty 作为底层，学好 Netty 不仅可以让我们自己实现高性能服务器，也能更好地理解其他的框架应用、阅读源码。

知识

- IO 模型（BIO / NIO）
- Channel
- Buffer
- Selector
- Netty 模型
- WebSocket 编程（动手做个聊天室）
- 相关技术：Vertx（中文文档：<http://vertxchina.github.io/vertx-translation-chinese/>，比 Netty 简单多了，实在看不懂 Netty 也可以学习下这个）

学习建议

Netty 是个相对比较难的技术，不像之前学的 SSM 框架那么容易上手。

Netty 还是需要一定学习成本的，一方面是国内资源相对缺乏，另一方面很多重要的概念（比如 NIO、Reactor 模型）比较抽象，要多动手写代码调试才能理解。第一遍看不懂很正常，不要气馁。

对于大多数同学来说，建议先从视频入门，了解 Netty 的基本用法和核心概念就行，不建议在 Netty 上花太多时间。因为在实际工作中，直接用到 Netty 的场景并不多（除非你做的是框架开发、IM 系统之类的），面试的时候一般也就考察一些 Netty 背后的思想（比如 NIO、零拷贝）而非框架本身的语法细节。

建议做个小项目，比如可以跟着教程做一个简单的 WebSocket 聊天室，这样对 Netty 的理解会更深刻。

经典面试题

1. Netty 有哪些优点？
2. 什么是 NIO？
3. 介绍 Netty 的零拷贝机制

资源

- ★ [尚硅谷 Netty 教程](#)：系统讲解 Netty
- 《Netty 实战》：经典书籍

○ 微服务（60 天）

关于微服务的详细学习，可以查看 [Spring Cloud 微服务学习路线](#)

随着互联网的发展，项目越来越复杂，单机且庞大的巨石项目已无法满足开发、运维、并发、可靠性等需求。

因此，后台架构不断演进，可以将庞大的项目拆分成一个个职责明确、功能独立的细小模块，模块可以部署在多台服务器上，相互配合协作，提供完整的系统能力。

换言之，想做大型项目，这块儿一定要好好学！

知识

Dubbo

- 架构演进
- RPC
- Zookeeper
- 服务提供者
- 服务消费者
- 项目搭建
- 相关技术：DubboX（对 Dubbo 的扩展）

○ 微服务

- 微服务概念
- Spring Cloud 框架
 - 子父工程
 - 服务注册和发现
 - 注册中心 Eureka、Zookeeper、Consul
 - Ribbon 负载均衡
 - Feign 服务调用
 - Hystrix 服务限流、降级、熔断
 - Resilience4j 服务容错
 - Gateway（Zuul）微服务网关
 - Config 分布式配置中心
 - 分布式服务总线
 - Sleuth + Zipkin 分布式链路追踪
- Spring Cloud Alibaba
 - Nacos 注册、配置中心
 - OpenFeign 服务调用

- Sentinel 流控
- Seata 分布式事务

接口管理

- Swagger 接口文档
- Postman 接口测试
- 相关技术: YApi、ShowDoc

学习建议

微服务这块内容听起来很高大上，但其实学起来没那么难，不要被那些专业术语吓到了。

先说说学习路径：时间不急的话，建议先从 Dubbo 学起，对分布式、RPC、微服务有些基本的了解，再去食用 Spring Cloud 全家桶会更香。学完 Spring Cloud 后，再去学 Spring Cloud Alibaba 就很简单了，因为很多概念都是相通的。

对于微服务的学习，原理 + 实践都很重要，但也不要被各种高大上的词汇唬住了。什么注册中心、配置中心、服务网关，听起来很复杂，其实都是上层（应用层）的东西，基本没有什么算法，跟着视频教程学，把项目跑起来，其实还是很好理解的。

记住，先会用再深入！

分布式相关知识非常多，但这里不用刻意去背，先通过视频教程实战使用一些微服务框架，把项目跑起来，对各个组件有个直观的认识就可以了。等到面试前，再去深入学习原理和细节。

大厂面试的时候很少问 Spring Cloud 框架的细节（比如某个配置怎么写），更多的是微服务以及各组件的一些思想，比如为什么需要注册中心、网关有什么好处、如何保证分布式系统的一致性等。所以学的时候，要多思考“为什么需要这个组件，它解决了什么问题”。

经典面试题

1. 什么是微服务，有哪些优缺点？
2. 什么是注册中心，能解决什么问题？

更多微服务面试题：

- [SpringCloud 面试题 - 面试鸭](#)
- [Dubbo 面试题 - 面试鸭](#)
- [Zookeeper 面试题 - 面试鸭](#)

资源

- ★ [API 开放平台 - 鱼皮原创](#)：运用 Dubbo 微服务和 Gateway 网关
- ★ [OJ 判题系统 - 鱼皮原创](#)：单体项目改造为微服务架构
- ★ [2025 尚硅谷 SpringCloud 最新教程](#)：2025 最新版，从入门到大牛，内容全面
- ★ [黑马 Spring Cloud 视频教程](#)：11 小时，非常凝练，适合快速入门
- ★ [尚硅谷 Dubbo 教程](#)：RPC 框架入门必看
- [尚硅谷 SpringCloud \(H版&alibaba\) 框架开发教程](#)：对比讲解，内容更全面
- [Apache Dubbo 官方文档](#)
- [Spring Cloud Alibaba 官方文档](#)
- [Swagger 教学文档](#)：跟着快速开始直接用就好了

○ 容器（7 天）

使用容器技术的好处可太多了！

将应用和环境进行封装，相互隔离、独立部署。不仅能够提高安全性、提高开发和维护效率，还能便于实现微服务、持续集成和交付。

关于 Docker 容器化的详细学习，可以查看 [Docker 容器化学习路线](#)；关于 Kubernetes 容器编排的详细学习，可以查看 [Kubernetes 学习路线](#)。

知识

- **Docker**
 - 容器概念
 - 镜像
 - 部署服务
 - Dockerfile
 - Docker Compose
 - Docker Machine
 - Docker Swarm
 - 多阶段构建
- **K8S (Kubernetes)**
 - K8S 架构
 - 工作负载
 - 资源类型
 - Pod
 - Pod 生命周期
 - Pod 安全策略
 - K8S 组件
 - K8S 对象
 - 部署应用
 - 服务
 - Ingress
 - Kubectl 命令行
 - 集群管理
- 相关技术：Apache Mesos、Mesosphere

学习建议

容器技术（Docker）是现在后端开发的标配，一定要学！但是 K8S 可以先不急。

Docker 真的很实用，能让你的项目部署变得非常简单。不用再担心 "在我电脑上能跑，在服务器上跑不起来" 的问题了。建议跟着视频教程，把 Docker 的基本操作（拉取镜像、运行容器、编写 Dockerfile）都实践一遍。然后试着用 Docker 部署你的项目，感受一下容器化的好处。

K8S (Kubernetes) 是容器编排平台，学习成本比较高。对于刚入门的同学来说，可以先不学，等工作后有需要再学也不迟。实际工作中，企业一般都有现成的 K8S 平台或者运维团队负责，开发同学只需要会用 Docker 就够了。

容器技术面试考察得不多，主要是考察你是否会用 Docker。所以不用花太多时间，会基本操作就行。

经典面试题

1. 什么是容器？
2. 使用 Docker 有哪些好处？
3. 如何快速启动多个 Docker 节点？

更多容器面试题：

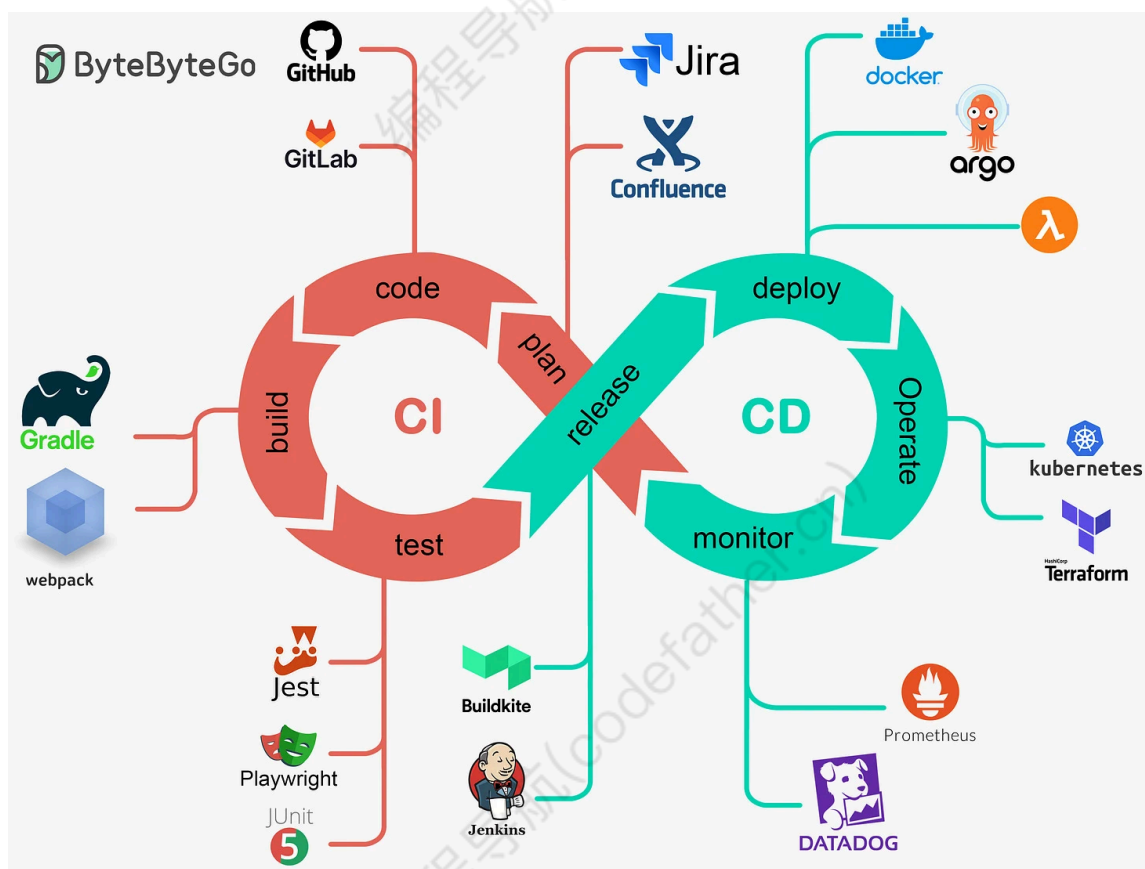
- [Docker 面试题 - 面试鸭](#)
- [Kubernetes 面试题 - 面试鸭](#)

资源

- [★ OJ 判题系统 - 鱼皮原创](#): 项目中运用 Docker 实现代码沙箱
- [★ 狂神说 - Docker 最新超详细视频教程](#): 通俗易懂
- [K8S 视频教程](#)
- [《深入浅出 Docker》](#)
- [Docker — 从入门到实践](#)
- [Docker 官方文档](#)
- [Docker 中文社区](#): 有很多 Docker 技术文章和学习笔记
- [Docker Hub](#): Docker 镜像仓库

🕒 CI / CD (3 天)

CI / CD 是指 **持续集成 / 持续交付**，贯穿整个研发到项目上线的过程，提高效率。



大公司一般都有自己的 CI / CD 平台。

知识

- 什么是 CI / CD
- CI / CD 有什么好处
- 使用任一 CI / CD 平台
- 相关技术: Jenkins、GitLab、微信云托管

学习建议

了解它是什么，并且实战使用任一 CI / CD 平台，感受它和传统开发运维到底有什么不同，就足够了。其实很简单，不要花太多时间。

真正要自己去搭建的时候，跟着官方文档来就行。

资源

- [尚硅谷 - Jenkins 持续集成工具教程](#)：实在要用到 Jenkins 再去学
- [Jenkins 官方文档](#)：有很多案例，要用的时候查一下就行
- ★ [微信云托管](#)
- [前端托管 Webify](#)：鱼皮有[视频教程](#)

练手项目

学习完框架后，即可跟着鱼皮的原创项目教程系列边学边做项目。用项目驱动学习，更快地掌握后端必学技术，并直接写在简历上：

- ★ [项目实战 - 鱼皮原创项目教程系列](#)（保姆级全栈项目，带简历写法和面试题）

其他开源项目：

- [miaosha](#)（秒杀系统设计和实现）
- [Mall](#)（包括前台中台系统及后台管理系统，基于 SpringBoot+MyBatis 实现）
- [paascloud-master](#)（基于 Spring Cloud + Vue + OAuth2.0 前后端分离商城系统）
- [mall-swarm](#)（微服务商城系统）
- [SecKill](#)（基于 SpringBoot+Mybatis+Redis+RabbitMQ 秒杀系统）
- [jeecg-boot](#)（低代码开发平台）
- [PassJava-Platform](#)（面试刷题的 Spring Cloud 开源系统）

尾声

看到这里，相信你已经感叹：编程语言一辈子学不完了！

但是，不用担心，通过对这么多知识点的学习，相信你已经有了的积累，也不自觉地锻炼了自主学习能力、资源检索能力、代码阅读能力、问题解决能力，之后的学习会越来越轻松。

接下来，可以试着用你学到的技术来解决实际的问题，自主从 0 开始做一些项目，保持编程手感。

阶段 5：项目实战

目标

综合所学技术从 0 到 1 开发和上线一个全面、有特色的、可以写进简历的个人项目。

学习建议

这个阶段是最关键的，能不能找到工作，很大程度上取决于你的项目经历。

为什么要做项目？

其实在之前的框架学习视频中应该就做过几个项目了，但相对不够完整和体系化。要想在简历上写得出彩，一定要有 1-2 个拿得出手的完整项目，最好是从需求分析、数据库设计、接口开发、前端页面到部署上线的完整流程都经历过。

如何选择项目？

- 1) 有想法的同学：可以做任何自己想做的项目，推荐参加一些作品类竞赛（比如中国软件杯、互联网+），练手、拿奖、收获项目经历一举三得。
- 2) 暂时没想法的同学：强烈建议先跟着 [鱼皮的原创项目教程系列](#) 做，从 0 到 1 保姆级带你做完整项目，还会教你怎么写简历、怎么应对面试。做完 2-3 个项目后，你就知道怎么独立做项目了。

3) 有一定基础的同学：可以试着用 GitHub 上的优质开源项目源码来学习，但是要注意选择 **较新的、有配套文档的、star 数较高的** 项目。

一般要做几个项目？

建议至少完整做 2 个项目，一个简单的（比如管理系统），一个稍微复杂的（比如包含微服务、缓存、消息队列等技术的项目）。不要贪多，2 个精品项目胜过 10 个半成品。

注意，必须要从 0 到 1 自己手写！

不要只是下载源码跑一跑就完事了，那样面试的时候会露馅。做项目的时候，建议边做边记录，把遇到的问题和解决方案都记下来，面试的时候能派上大用场。

下面推荐一些优质的项目实战视频教程和开源项目源码。

视频教程

强烈推荐 [鱼皮原创项目教程系列](#)，[编程导航](#) 鱼友可学。

阶段 1 - 新手入门（适合刚学完框架的同学）：

1. [用户中心项目](#)：保姆级前后端完整项目教程，系统学习项目开发流程和上线方法
2. [伙伴匹配系统](#)：移动端网站，学习 Redis、事务、并发编程等后端知识
3. [AI 编程助手](#)：适合新手入门 AI 应用开发，实战 LangChain4j 框架

阶段 2 - 真实业务实战（业务完整，极具实用价值）：

4. [AI 零代码应用生成平台](#)：对标大厂，实战 AI 智能体、微服务架构、多种设计模式
5. [智能协同云图库](#)：实战文件存管、权限控制、实时协同等企业级场景
6. [AI 答题应用平台](#)：深入业务场景，学习分库分表、分布式锁、RxJava 响应式编程
7. [智能面试刷题平台](#)：实战 Redis 多级缓存、Elasticsearch、Sentinel 流控
8. [代码生成器共享平台](#)：学习命令行开发、模板引擎、Vert.x、设计模式

阶段 3 - 快速补充技术栈（小而精，侧重某一项技术）：

9. [AI 超级智能体项目](#)：学习 AI 应用开发，掌握新时代程序员必知的 AI 技术
10. [聚合搜索平台](#)：学习爬虫 + Elastic Stack + 设计模式
11. [智能 BI 项目](#)：学习线程池 + RabbitMQ 消息队列 + AI 应用开发

阶段 4 - 技术进阶（涉及架构设计，更侧重技术提升）：

12. [API 开放平台](#)：大厂业务，学习 SDK 开发 + API 签名认证 + Dubbo + Gateway 网关
13. [OJ 判题系统](#)：学习多种设计模式 + 微服务改造 + Docker 代码沙箱
14. [手写 RPC 框架](#)：从 0 到 1 开发轮子，大幅提升架构设计能力

更多项目详情和学习建议：[鱼皮项目学习建议（必读）](#)

公开视频教程：

- [尚硅谷 - 谷粒学院 - 微服务 + 全栈 - 在线教育实战项目](#)：全栈项目，前后端讲得都很全面
- [尚硅谷 - 尚筹网 - SSM 框架 + 微服务架构](#)：500 多集，包含完整的用户权限管理
- [黑马 - 24 小时搞定 Java 毕设电商项目](#)
- [黑马 Java 项目《传智健康》](#)：超完整的企业级医疗行业项目
- [黑马 Java 项目《万信金融》](#)：企业级开发实战
- [黑马 Java 项目《iHRM 人力资源管理系统》](#)：SaaS 移动办公
- [黑马 Java 项目《好客租房》](#)

常用类库

工具类

- [Guava](#): 谷歌开发的 Java 工具库
- [Apache Commons](#): 各类工具库, 比如 commons-lang、commons-io、commons-collections 等
- [Hutool](#): Java 工具集库
- [Lombok](#): Java 增强库, 自动生成 getter/setter
- [Apache HttpClient](#): HTTP 客户端库
- [OkHttp](#): 适用于 JVM、Android 等平台的 Http 客户端
- [Gson](#): 谷歌的 JSON 处理库
- [Jcommander](#): Java 命令行参数解析框架
- [Apache PDFBox](#): PDF 操作库
- [EasyExcel](#): 阿里的 Excel 处理库
- [Apache POI](#): 表格文件处理库

测试类

- [JUnit](#): Java 测试框架
- [Mockito](#): Java 单元测试 Mock 框架
- [Selenium](#): 浏览器自动化框架
- [htmlunit](#): Java 模拟浏览器
- [TestNG](#): Java 测试框架
- [Jacoco](#): Java 代码覆盖度库

其他实用库

- [cglib](#): 字节码生成库
- [Arthas](#): Java 诊断工具
- [config](#): 针对 JVM 的配置库
- [Quasar](#): Java 线程库
- [drools](#): Java 规则引擎
- [Caffeine](#): Java 高性能缓存库
- [Disruptor](#): 高性能线程间消息传递库
- [Knife4j](#): Swagger 文档增强
- [Thumbnailator](#): Java 缩略图生成库
- [Logback](#): Java 日志库
- [Apache Camel](#): 消息传输集成框架
- [Quartz](#): 定时任务调度库
- [Apache Mahout](#): 机器学习库
- [Apache OpenNLP](#): NLP 工具库
- [RxJava](#): JVM 反应式编程框架
- [JProfiler](#): 性能分析库
- [jsoup](#): HTML 文档解析库
- [webmagic](#): Java 爬虫框架

IDEA 插件

工欲善其事, 必先利其器。IDEA 拥有丰富的插件生态, 合理使用插件能够显著提升开发效率。

这里精选一些必装的插件, 更多 IDEA 插件推荐请查看:

- ★ 完整的 IDEA 插件推荐文档: [IDEA 插件精选推荐](#)
- ★ 鱼皮的 IDEA 插件推荐视频: <https://www.bilibili.com/video/BV1yb4y1a7Aq>

精选必装插件

必装插件

- [Chinese Language Pack](#) - 中文语言包
- [Translation](#) - 翻译插件，鼠标选中文本右键翻译
- [Key Promoter X](#) - 快捷键提示插件，帮你养成使用快捷键的习惯
- [Rainbow Brackets](#) - 彩虹括号，通过颜色区分括号层级
- [CodeGlance](#) - 代码小地图，方便快速定位

效率提升插件

- [String Manipulation](#) - 字符串快捷处理
- [GsonFormatPlus](#) - 根据 JSON 生成对象
- [MyBatisX](#) - MyBatis 增强插件，自动生成代码
- [Tabnine AI Code Completion](#) - AI 代码补全
- [JUnitGenerator V2.0](#) - 自动生成单元测试
- [RestfulTool](#) - 辅助 Web 开发的工具集
- [SequenceDiagram](#) - 自动生成方法调用时序图

代码质量插件

- [CheckStyle-IDEA](#) - 自动检查 Java 代码规范
- [Alibaba Java Coding Guidelines](#) - 阿里巴巴代码规范检查插件
- [SonarLint](#) - 发现和修复代码的错误和漏洞

常用软件

开发相关

- [JetBrains IDEA](#): Java 集成开发环境，凭学生邮箱可申请免费使用
- [Visual Studio Code](#): 插件化代码编辑器，轻量且功能强大
- [Sublime Text](#): 轻量代码编辑器
- [Navicat](#): 数据库管理软件，支持多种数据库
- [JMeter](#): Java 性能测试工具
- [JVisual VM](#): Java 运行状态可视化工具
- [XShell](#): SSH 连接软件
- [XFtp](#): FTP 连接软件
- [Redis Desktop Manager](#): Redis 可视化管理工具
- [Postman](#): 接口测试工具
- [VMware](#): 虚拟机软件
- [Chocolatey](#): Windows 软件包管理器
- [Typora](#): Markdown 写作软件

效率工具

- [剪切助手](#): 强大的剪切板管理工具
- [uTools](#): 插件化的效率工具
- [XMind](#): 思维导图软件
- [Qdir](#): Windows 多窗口文件管理器

项目源码 (50 套)

鱼皮原创项目

- ★ [项目实战 - 鱼皮原创项目教程系列](#) (强烈推荐)
- [SQL 数据生成器](#) (React + Java)
- [结构化 SQL 语句生成器](#)
- [AI 自动回复工具](#) (Java 项目)
- [表情包网站](#) (Vue + Java)

电商秒杀

- 天猫整站 J2EE: <https://how2j.cn/module/115.htm>
- 天猫整站 SSM: <https://how2j.cn/module/134.html>
- 天猫整站 Springboot: <https://how2j.cn/module/156.html>
- SpringBoot 电商商城系统 Mall4j: <https://github.com/gz-yami/mall4j>
- SpringBoot 完整电商系统 Mall: <https://github.com/macrozheng/mall> (包括前台商城系统及后台管理系统, 基于 SpringBoot+MyBatis 实现)
- newbee-mall: <https://github.com/newbee-ltd/newbee-mall> (一套电商系统, 包括 newbee-mall 商城系统及 newbee-mall-admin 商城后台管理系统, 基于 Spring Boot 2.X 及相关技术栈开发)
- paascloud-master: <https://github.com/paascloud/paascloud-master> (基于 spring cloud + vue + oAuth2.0, 前后端分离商城系统)
- mall-swarm: <https://github.com/macrozheng/mall-swarm> (一套微服务商城系统, 采用了 Spring Cloud Greenwich、Spring Boot 2、MyBatis、Docker、Elasticsearch 等核心技术, 同时提供了基于 Vue 的管理后台方便快速搭建系统)
- onemall: <https://github.com/YunaiV/onemall> (mall 商城, 基于微服务的思想, 构建在 B2C 电商场景下的项目实战。核心技术栈, 是 Spring Boot + Dubbo。未来, 会重构成 Spring Cloud Alibaba)
- litemall: <https://github.com/linlinjava/litemall> (又一个商城, litemall = Spring Boot 后端 + Vue 管理员前端 + 微信小程序用户前端 + Vue 用户移动端)
- xmall: <https://github.com/Exrick/xmall> (基于 SOA 架构的分布式电商购物商城 前后端分离 前台商城:Vue 全家桶 后台管理系统)
- miaosha: <https://github.com/qiurunze123/miaosha> (秒杀系统设计和实现)
- SecKill: <https://github.com/hfbin/Seckill> (基于 SpringBoot+Mybatis+Redis+RabbitMQ 秒杀系统)

博客论坛

- [Mblog](#): 开源 Java 博客系统
- [halo](#): 一个优秀的开源博客发布应用
- [forum-java](#): 一款用 Java (spring boot) 实现的现代化社区 (论坛/问答/BBS/社交网络/博客) 系统平台
- [vhr](#): 微人事是一个前后端分离的人力资源管理系统, 项目采用 SpringBoot+Vue 开发。
- [favorites-web](#): 云收藏 Spring Boot 2.X 开源项目。云收藏是一个使用 Spring Boot 构建的开源网站, 可以让用户在线随时随地收藏的一个网站, 在网站上分类整理收藏的网站或者文章。
- [community](#): 码问, 开源论坛、问答系统, 现有功能提问、回复、通知、最新、最热、消除零回复功能。技术栈 Spring、Spring Boot、MyBatis、MySQL/H2、Bootstrap
- [NiterForum](#): 尼特社区-NiterForum-一个论坛/社区程序。后端Springboot/MyBatis/Maven/MySQL, 前端Thymeleaf/Layui。可供初学者, 学习、交流使用。
- [VBlog](#): V部落, Vue+SpringBoot实现的多用户博客管理平台!
- [NiceFish](#): SpringBoot/SpringCloud 前后端分离项目
- [My-Blog](#): My Blog 是由 SpringBoot + Mybatis + Thymeleaf 等技术实现的 Java 博客系统, 页面美观、功能齐全、部署简单及完善的代码。
- [My-Blog-layui](#): layui 版本的 My-Blog, 由 SpringBoot + Layui + Mybatis + Thymeleaf 等技术实现的 Java 博客系统。
- [symphony](#): Java 实现的现代化社区

管理系统

- [Spring-Cloud-Admin](#): Cloud-Admin 是国内首个基于 Spring Cloud 微服务化开发平台, 具有统一授权、认证后台管理系统, 其中包含具备用户管理、资源权限管理、网关 API 管理等多个模块, 支持多业务系统并行开发, 可以作为后端服务的开发脚手架。代码简洁, 架构清晰, 适合学习和直接项目中使用。核心技术采用 Spring Boot2 以及 Spring Cloud Gateway 相关核心组件, 前端采用 vue-element-admin 组件。
- [bootshiro](#): 基于 springboot+shiro+jwt 的资源无状态认证权限管理系统后端
- [悟空CRM](#): 基于jfinal+vue+ElementUI的前后端分离CRM系统
- [EL-ADMIN](#): 基于 SpringBoot 的后台管理系统

- [pig](#): 基于 Spring Boot 2.2、Spring Cloud Hoxton & Alibaba、OAuth2 的 RBAC 权限管理系统。
- [Spring Boot-Shiro-Vue](#): 基于Spring Boot-Shiro-Vue 的权限管理
- [studentmanager](#): 基于springboot+mybatis学生管理系统
- [jshERP](#): 华夏ERP基于SpringBoot框架和SaaS模式，立志为中小企业提供开源好用的ERP软件，目前专注进销存+财务功能。主要模块有零售管理、采购管理、销售管理、仓库管理、财务管理、报表查询、系统管理等。支持预付款、收入支出、仓库调拨、组装拆卸、订单等特色功能。拥有库存状况、出入库统计等报表。同时对角色和权限进行了细致全面控制，精确到每个按钮和菜单。
- [HotelSystem](#): 酒店管理系统 Java,tomcat,mysql,servlet,jsp实现，没有使用任何框架

开发平台

- [open-capacity-platform](#): 微服务能力开发平台
- [jeecg-boot](#): JeecgBoot是一款基于BPM的低代码平台！前后端分离架构 SpringBoot 2.x, SpringCloud, Ant Design&Vue, Mybatis-plus, Shiro, JWT, 支持微服务。强大的代码生成器让前后端代码一键生成，实现低代码开发！

其他

- [学之思在线考试系统](#): 一款 java + vue 的前后端分离的考试系统
- [PassJava-Platform](#): 一款面试刷题的 Spring Cloud 开源系统
- [kkFileView](#): 使用spring boot打造文件文档在线预览项目
- [dynamic-datasource](#): 一个基于springboot的快速集成多数据源的启动器
- [moti-cloud](#): 莫提网盘，基于 SpringBoot+MyBatis+Thymeleaf+Bootstrap, 适合初学者
- [threadandjuc](#): three-high-import 高可用\高可靠\高性能，三高多线程导入系统（该项目意义为理论贯通）
- [proxyee-down](#): http下载工具，基于http代理，支持多连接分块下载
- [hosp_order](#): 医院预约挂号系统，基于 SSM 框架
- [趋势投资 SpringCloud](#)
- [DiyTomcat](#)

阶段 6: Java 高级

目标

不满足于能做，而是通过更 **深入** 和 **广泛** 的学习，实现高质量的代码和更优秀的架构，培养解决问题的能力。

到了这个阶段，你已经不是 Java 新手了，而是一个能独立做项目的开发者。但是如果想在进大厂、拿高薪，或者想在技术上有更深的造诣，就需要继续深入学习。

这个阶段的学习，不能只是被动地看教程了。建议除了看完整的教程外，平时多自主搜索信息去学习，积少成多。比如遇到了一个不了解的名词（比如虚拟线程、ZGC），可以去网上搜一下，感兴趣的话再进行下一步的学习。慢慢地，你的知识面就会越来越广。

🕒 并发编程 (21 天)

对 Java 后端开发工程师来说，懂得如何利用有限的系统资源来提高系统的性能是很重要的，也是大厂面试考察的重点，因此并发编程（尤其是 Java 并发包的使用）这块的知识很重要。

把它放到高级，是因为在学并发编程前，需要有一定的编程经验、了解一定的操作系统知识。

知识

传统并发编程【必学】:

- 线程和进程
- 线程状态
- 并行和并发

- 同步和异步
- Synchronized
- Volatile 关键字
- Lock 锁
- 死锁
- 可重入锁
- 线程安全
- 线程池
- JUC 的使用
- AQS
- Fork Join
- CAS

新特性【了解】:

- **虚拟线程 (Virtual Threads)**: Java 21 引入的重要特性, 是轻量级的线程, 可以极大提升并发性能。虚拟线程让我们能够以更低的资源开销创建数百万个线程, 特别适合 I/O 密集型应用。感兴趣的同学可以查看 [Java 21 新特性](#)。
- **结构化并发 (Structured Concurrency)**: 让并发代码更易于理解和维护

学习建议

并发编程入门不难, 依然是 **先学会使用** 基础的 Java 并发包, 再通过大量地实践和测试, 了解一些原理, 才能真正掌握何时使用、如何更合理地使用并发编程。而不是张口闭口多线程, 上天入地高并发。

对于虚拟线程这类新特性, 初学者先不用着急学, 等你掌握了传统的并发编程知识后, 再去了解会更有收获。但如果你的项目中有大量的 I/O 操作 (比如网络请求、数据库查询), 了解虚拟线程能帮你写出性能更好的代码。

经典面试题

1. volatile 关键字的作用
2. 使用线程池有哪些好处?
3. 线程池参数如何设置?
4. 什么是线程安全问题, 如何解决?
5. 介绍 synchronized 的锁升级机制
6. CopyOnWriteArrayList 适用于哪种场景?

资源

- ★ [伙伴匹配系统 - 鱼皮原创](#): 项目中运用并发编程解决实际问题
- ★ [智能 BI 项目 - 鱼皮原创](#): 运用线程池、并发编程
- ★ [OJ 判题系统 - 鱼皮原创](#): 运用并发编程
- ★ [尚硅谷 - 大厂必备技术之 JUC 并发编程 2021 最新版](#): 短、精、新
- [黑马程序员 - 全面深入学习 Java 并发编程](#): 讲得很细、全面深入
- 《Java 并发编程实战》: 国外经典书籍
- 《Java 并发编程艺术》: 国人写的, 理论思想内容较多, 有时间建议反复看
- [《图解 Java 多线程设计模式》](#): 提取码: 9gc7, 可以额外学习多线程设计模式
- [Java 并发知识点总结](#)

🕒 JVM (30 天)

想要深入理解 Java, 探秘 Java 跨平台的奥秘, 一定要了解 Java 底层的虚拟机技术。

了解虚拟机、掌握虚拟机性能调优方法, 有助于你写出更高性能、资源占用更小的优质程序。

在学习 JVM 的过程中, 也能学到很多精妙的设计, 开拓思路。

知识

- JVM 内存结构
- JVM 生命周期
- 主流虚拟机
- Java 代码执行流程
- 类加载
 - 类加载器
 - 类加载过程
 - 双亲委派机制
- 垃圾回收
 - 垃圾回收器
 - 垃圾回收策略
 - 垃圾回收算法
 - StopTheWorld
- 字节码
- 内存分配和回收
- JVM 性能调优
 - 性能分析方法
 - 常用工具
 - 参数设置
- Java 探针
- 线上故障分析

学习建议

JVM 是 Java 的灵魂，想要深入理解 Java、写出高性能的代码，JVM 是必学的。但是 JVM 的知识确实有点枯燥，不像框架那样能立刻做出东西来。

建议先看视频入门，有实操的地方一定要实操！比如分析 GC 日志、使用 JVisualVM 查看内存使用情况、通过参数调优等。光看理论很难理解，自己动手分析过一次，印象会深刻很多。

第一遍看不懂很正常，JVM 涉及很多底层知识。可以先快速过一遍视频，有个大致印象，再去看书（比如《深入理解 Java 虚拟机（第三版）》）巩固，最后面试前再刷一遍，这样效果会好很多。我当时就是看了三遍才真正理解的。

注意，不同目标有不同的学法：

- 如果只是为了通过面试，可以直接看更精简的视频（比如狂神的 JVM 快速入门），重点背一下常见面试题就行。
- 如果想真正学好 JVM、做性能调优，那《深入理解 Java 虚拟机（第三版）》一定要读，而且要边读边实践。

经典面试题

1. 介绍 JVM 的内存模型？
2. JVM 内存为什么要分代？
3. 介绍一次完整的 GC 流程
4. 介绍双亲委派模型，为什么需要它？

资源

- [★ 尚硅谷宋红康-JVM 全套教程详解](#)：讲得相当全面！附有实操
- [狂神说 Java - JVM 快速入门篇](#)：讲得有点浅，但都是面试重点，时间紧可以直接看这个
- 《深入理解 Java 虚拟机（第三版）》：有理论有实践，内容丰富，JVM 学习神书
- [Java 虚拟机底层原理知识总结](#)
- [阿里云 JVM 实战](#)

- [Arthas 开源 Java 诊断工具](#)

○ Java 高级知识

通过阅读文章了解即可

知识

- 动态代理
- Java 探针
- 字节码
- Unsafe 类
- 协程 / 纤程

架构设计

○ 分布式

- 分布式理论
 - CAP
 - BASE
- 分布式缓存
 - Redis
 - Memcached
 - Etcd
- 一致性算法
 - Raft
 - Paxos
 - 一致性哈希
- 分布式事务
 - 解决方案
 - 2PC
 - 3PC
 - TCC
 - 本地消息表
 - MQ 事务消息
 - 最大努力通知
 - [LCN 分布式事务框架](#)
- 分布式 id 生成
 - [Leaf 分布式 id 生成服务](#)
- 分布式任务调度
 - [XXL-JOB 调度平台](#)
 - [elastic-job](#)
- 分布式服务调用
 - trpc
- 分布式存储
 - HDFS
 - Ceph
- 分布式数据库
 - TiDB

- OceanBase
- 分布式文件系统
 - HDFS
- 分布式协调
 - Zookeeper
- 分布式监控
 - Prometheus
 - Zabbix
- 分布式消息队列
 - RabbitMQ
 - Kafka
 - Apache Pulsar
- 分布式日志收集
 - Elastic Stack
 - Loki
- 分布式搜索引擎
 - Elasticsearch
- 分布式链路追踪
 - Apache SkyWalking
- 分布式配置中心
 - Apollo
 - Nacos

❶ 高可用

- 限流
- 降级熔断
- 冷备
- 双机热备
- 同城双活
- 异地双活
- 异地多活
- 容灾备份

❷ 高并发

- 数据库
 - 分库分表
 - MyCat 中间件
 - Apache ShardingSphere 中间件
 - 读写分离
- 缓存
 - 缓存雪崩
 - 缓存击穿
 - 缓存穿透
- 负载均衡
 - 负载均衡算法
 - 软硬件负载均衡（2、3、4、7 层）

● 服务网格

服务网格用来描述组成应用程序的微服务网络以及它们之间的交互。服务网格的规模和复杂性不断增长，它将会变得越来越难以理解和管理，常见的需求包括服务发现、负载均衡、故障恢复、度量和监控等。

知识

- Istio
 - 流量管理
 - 安全性
 - 可观测性
- Envoy（开源的边缘和服务代理）

资源

- istio 官方文档: <https://preliminary.istio.io/latest/zh>
- istio 视频教程: <https://www.bilibili.com/video/BV1Lf4y1x7j8>

● DDD 领域驱动设计

将数据、业务流程抽象成容易理解的领域模型，通过用代码实现领域模型，来组成完整的业务系统。

知识

- DDD 的优势
- DDD 的适用场景
- DDD 核心概念
 - 领域模型分类：失血、贫血、充血、涨血
 - 子域划分：核心域、通用域、支撑域
 - 限界上下文
 - 实体和值对象
 - 聚合设计
 - 领域事件
- DDD 实践

资源

- ★ [智能协同云图库项目 - 鱼皮原创](#)：实战 DDD 领域驱动设计开发企业级项目
- [DDD 资源大全](#)

● 其他

- Sidecar
- Serverless
- 云原生

学习建议

架构设计的学习没有顶点，多看文章，思考每种设计的优缺点和适用场景，有机会的话在企业中实践即可。

还在学校、或者初入这行的同学切记，千万不要一味地去背诵架构设计的八股文。你可以背，但是这一块的知识只有结合具体的项目才有意义，所以要多做项目去实践设计的合理性，而不是什么设计都咔咔往系统里去怼。比如面试问到分布式事务，能结合自己项目中用分布式事务解决问题的经验去回答最好。

● 其他技术

- 热数据探测技术：京东 HotKey
- 数据库流水订阅：阿里 Canal
- 监控告警
- 应用安全
- 故障演练

- 流量回放

阶段 7：Java 求职

目标

一句话：找到好工作

学习建议

找工作是一个系统工程，不是简单地投简历、面试就行了。下面分享一些我的经验和建议：

1) 尽早做规划，可以通过大厂招聘官网的岗位描述来了解岗位的要求，看看自己还欠缺哪些技能。**建议至少提前 3 个月开始准备**，不要等到要找工作了才开始慌。

推荐阅读鱼皮的 [保姆级求职指南](#)，从求职规划到拿 Offer 全流程讲解。

2) 打磨简历：简历是你的第一张名片，一份好的简历能让你获得更多面试机会。建议使用 [老鱼简历](#) 制作简历，有大量专业简历模板。



关于如何写好简历，推荐学习鱼皮的 [保姆级写简历指南](#)。还可以查看 [真实简历案例](#)，看看别人的简历是怎么写的。

3) 多刷面试题：建议使用 [面试鸭](#) 刷题，支持按公司、题型、难度筛选。



如果要进大厂的话，还需要坚持刷算法题（LeetCode），每天 1-2 道，保持手感。

4) 多看面经和面试视频：

- 编程导航整理了大量 [真实面经](#)，看看别人都被问了什么
- 观看 [几百场真实面试视频](#)，学习面试技巧
- 关注 [面试鸭 B 站账号](#)，看面试题讲解视频
- 查看 [编程导航求职干货分享](#)

5) 多参加面试：不要等准备得很充分了才去面试，可以先拿一些不太想去的公司练手，积累面试经验。每次面试后都要复盘总结，记录下被问到的问题和自己答得不好的地方，下次改进。

还可以利用 AI 进行 [1 对 1 模拟面试](#)，消除真实面试时的紧张感。

资源

校招岗位

- 阿里 Java 开发：<https://campus.alibaba.com/position.htm?refno=12699>
- 腾讯后台开发：https://join.qq.com/post_detail.html?pid=1&id=101&tid=2
- 腾讯全栈开发：https://join.qq.com/post_detail.html?pid=1&id=137&tid=2
- 腾讯运营开发：https://join.qq.com/post_detail.html?pid=1&id=105&tid=2
- 美团后端开发：<https://campus.meituan.com/jobs?jobFamily=1&jobId=4005&jobType=1&pageNo=2>

社招岗位

- 阿里社招：<https://job.alibaba.com/zhaopin/positionList.htm>
- 腾讯社招：<https://careers.tencent.com/search.html>

实习

- 实习僧：<https://www.shixiseng.com/>

鱼皮经历

- ★ [我学计算机的四年，共勉](#)（从 0 开始的编程学习进大厂经历）

- [★ 我的第一份实习](#)
- [★ 我的第二份实习，字节跳动](#)

知识总结

- [阿里 Java 技术图谱](#)

面试题刷题

- [★ 面试鸭 - Java 面试题库](#) (支持按题目、题型、难度、公司搜索, 还有 AI 模拟面试)
- [★ 编程导航真实面经大全](#)
- [编程导航面经汇总](#)

面试题视频

更多面试题讲解视频, 请关注 [面试鸭 B 站账号](#)。

- [尚硅谷 2021 逆袭版 Java 面试题第三季](#) (讲解面试高频题)
- [阿里大佬透彻讲解 Java 面试 500 道必考题](#)

阶段 8: 持续学习

目标

持续追求技术的深度和广度, 培养自己的 **核心竞争力** 和 **不可替代性**, 学无止境!

学习建议

技术的学习永无止境, 无论你是刚入行的新人, 还是工作多年的老鸟, 都要保持学习的习惯。

- 1) 自主学习能力: 到了这个阶段, 要培养自主学习的能力。遇到不懂的技术, 要学会自己搜索资料、阅读官方文档、看源码。不要什么都等着别人来教你。
- 2) 多看技术博客: 关注一些优质的技术博客和技术团队 (比如美团技术团队、阿里技术团队), 看看大厂是怎么解决技术问题的, 能开拓思路。
- 3) 深度和广度并重: 既要在某个领域深耕 (比如精通 JVM 调优、精通并发编程), 也要拓展知识面 (了解前端、大数据、云原生等)。T 型人才更受欢迎。
- 4) 多实践, 多造轮子: 光看不练假把式, 要多动手实践。可以尝试自己实现一些常用的框架或工具, 比如实现一个简单的 ORM 框架、实现一个 RPC 框架, 这个过程会让你对技术的理解更深刻。
- 5) 多交流, 多分享: 加入一些技术社区 (比如 [编程导航](#)), 和其他开发者交流学习经验、分享技术心得。教是最好的学, 尝试写技术博客、做技术分享, 能帮你把知识真正内化。

学习方向

框架源码

- Spring
- SpringBoot
- SpringMVC
- MyBatis
- Netty
- Dubbo
- SpringCloud

计算机原理

- 《算法导论》: <https://www.bilibili.com/video/av48922404>

- 《现代操作系统》: <https://www.bilibili.com/video/av9555596>
- 《深入理解计算机系统》: <https://www.bilibili.com/video/av31289365>
- 《计算机网络: 自顶向下方法》: <https://www.bilibili.com/video/BV1JV411t7ow>
- 《计算机程序的构造和解释》: <https://www.bilibili.com/video/av8515129>
- 《数据库系统概论》: <https://www.bilibili.com/video/BV1G54y1d7ZK>

数据库 / 中间件 / 分布式

- 数据库
 - MySQL
 - PostgreSQL
- 缓存
 - Redis
- 队列
 - Apache Kafka
 - Apache Pulsar
- 搜索引擎
 - Elastic Stack
 - Elasticsearch
 - logstash
 - kibana
 - beats
- 容器
 - Docker
 - K8S

解决方案

- 广告系统
- 电商系统
- 搜索系统
- 支付转账
- 游戏后台
- 即时通讯
- 社交系统
- CMS 系统
- ERP 系统
- OA 系统
- 代码生成
- 权限管理
- 秒杀活动

架构设计

同阶段 6 的架构设计部分

大数据

- 5V 特点
- Hadoop
- HDFS
- MapReduce
- Spark

- Flink
- Storm
- Hive
- HBase
- Druid
- Kylin
- Pig
- Mahout

前沿技术

- 云原生: <https://www.jianshu.com/p/a37baa7c3eff>
 - Quasar Framework: <http://www.quasarchs.com/>
- 服务网格: <https://www.redhat.com/zh/topics/microservices/what-is-a-service-mesh>
 - istio: <https://github.com/istio/istio>
 - 官网: <https://www.graalvm.org/>
- ZGC: <https://mp.weixin.qq.com/s/ag5u2EP0bx7bZr7hkcrOTg> (新一代垃圾回收器)
 - 官网: <http://openjdk.java.net/projects/zgc/>

自学 Java 专题资源

- ★ 编程导航: <https://www.codefather.cn/> (学习路线、项目教程、面试题、编程资源一站式平台)
- ★ GitHub Java 专区: <https://github.com/topics/java>
- ★ GitHub Java 合集: <https://github.com/akullpp/awesome-java>
- StackOverflow: <https://stackoverflow.com/questions/tagged/java> (解决问题必备)
- Netflix TechBlog: <https://netflixtechblog.com/> (Netflix 微服务架构实践)
- Uber Engineering Blog: <https://www.uber.com/blog/engineering/> (Uber 大规模系统设计)
- Airbnb Tech Blog: <https://medium.com/airbnb-engineering> (Airbnb 技术实践)
- LinkedIn Engineering: <https://www.linkedin.com/blog/engineering> (LinkedIn 后端技术)
- 美团技术团队: <https://tech.meituan.com/>
- 有赞技术团队: <https://tech.youzan.com/tag/back-end/>

写在最后

看完这份学习路线，你可能会觉得：天呐，Java 要学的东西太多了，什么时候才能学完啊！

不要被吓到，这份路线涵盖了从零基础到高级的所有内容，不是所有的东西都要学完才能找工作的。根据你的目标和时间，可以有选择地学习：

- **如果你只有 6 个月**：学完阶段 1、3、5 就可以开始找工作了（语言基础 + 开发框架 + 项目实战）
- **如果你有 1 年时间**：建议学完阶段 1 ~ 5，再选择性地学习阶段 4 和阶段 6 的部分内容
- **如果你目标是大厂**：建议每个阶段都要学习，尤其是并发编程、JVM、微服务这些面试重点

记住，学习路线只是一个参考，不是死板的教条。每个人的情况不同，要根据自己的实际情况灵活调整。重要的是保持学习的热情，不断实践，持续进步。

最后，祝大家学习顺利，早日找到心仪的工作！如果在学习过程中遇到问题，欢迎来 [编程导航](#) 和大家一起交流讨论~

加油小伙伴们 🤝！



程序员必备资源

- 1) 程序员学习交流圈：[极客教程、实战项目、求职宝典](#)
- 2) 程序员面试八股文：[实习/校招/社招高频考点、企业真题解析](#)
- 3) 程序员写简历神器：[专业模板、丰富例句、直通面试](#)
- 4) AI 知识资源大全：[前沿技术、最新 AI 资讯、提示词大全](#)
- 5) 1 对 1 模拟面试：[实习/校招/社招面试拿 Offer 必备](#)